



Marco Antonio PINNA

Étudiant à la Plateforme de Cannes présente:

<https://cvmap.page.gd>

Son projet de soutenance pour le Titre de Développeur
Web et Web Mobile

StaffEase Pro *Une Solution Modulaire de Gestion des Plannings, des
Signatures Numériques et des Ressources Humaines pour le Secteur
Hôtelier et Touristique*

Sommaire

1. Introduction

- 1.1 Contexte et Motivation
 - 1.1.1 Contexte du Secteur Hôtelier et Problématiques Identifiées
 - 1.1.2 Gestion des Présences et des Signatures : Un Processus Peu Sécurisé
 - 1.1.3 Gestion des Absences et des Maladies : Un Processus Peu Structuré
- 1.2 Motivation Personnelle et Professionnelle
 - 1.2.1 Une Expérience de 24 Ans dans l'Hôtellerie
 - 1.2.2 Une Passion pour les Solutions Technologiques

2. Présentation du Projet : StaffEase Pro

- 2.1 StaffEase Pro : Une Réponse aux Besoins du Secteur
- 2.2 Une Première Étape vers une Solution Globale pour le Secteur Touristique
- 2.3 Une Solution pour le Futur

3. Conception

- 3.1 Méthodologies, Technologies et Outils Utilisés
 - 3.1.1 Méthodologies : Agile (Scrum), Merise, MVC
 - 3.1.2 Technologies : PHP 8.1, MySQL, JavaScript (ES6), HTML5/CSS3
 - 3.1.3 Outils : Figma, GitHub, MAMP, Excalidraw, MySQL Workbench
- 3.2 Premières Étapes et Difficultés Rencontrées
- 3.3 Utilisation de GitHub pour la Gestion de Projet

4. Design et Identité Visuelle

- 4.1 Identité Visuelle (Logo, Couleurs, Typographie)
- 4.2 Maquettage avec Figma
- 4.3 Composants Modulaires et Réutilisables
- 4.4 Responsive Design
- 4.5 Prédétermination des Maquettes pour Chaque Rôle

5. Base de Données

- 5.1 Modélisation avec Merise (MCD, MLD, MPD)
- 5.2 Implémentation en MySQL
- 5.3 Requêtes Clés et Optimisation
- 5.4 Notes sur les Tests Initiaux et les Ajustements

6. Architecture Technique

- 6.1 Architecture MVC
- 6.2 Front Controller et Routage
- 6.3 Intégration des Maquettes Figma en Code

7. Fonctionnalités Clés

- 7.1 Gestion des Utilisateurs (Rôles : Super Admin, Admin, Manager, Employé)
- 7.2 Gestion des Plannings (Shifts)
- 7.3 Signatures Numériques
 - 7.3.1 Génération d'un Lien Unique
 - 7.3.2 Vérification du Réseau
 - 7.3.3 Capture et Enregistrement de la Signature
- 7.4 Impression du Planning

8. Sécurité

- 8.1 Protection contre les Injections SQL (PDO)
- 8.2 Gestion des Sessions et des Rôles
- 8.3 Protection contre les Attaques XSS et CSRF

9. Performance, Accessibilité et SEO

- 9.1 Optimisation des Performances
- 9.2 Accessibilité (WCAG 2.1)
- 9.3 Référencement Naturel (SEO)

10. Tests et Déploiement

- 10.1 Stratégie de Test
- 10.2 Environnements de Test
- 10.3 Déploiement sur Infinity Free

11. Conclusion et Vision Future

- 11.1 Bilan du Projet
- 11.2 Impact pour les Utilisateurs et les Entreprises
- 11.3 Retour d'Expérience
- 11.4 Vision Future : Écosystème d'Applications Intégrées
- 11.5 Analyse Concurrentielle

12. Annexes

- Annexe 1 : Tableau de Suivi des Tests
- Annexe 2 : Exemples de Code Commentés
- Annexe 3 : Schémas Techniques
- Annexe 4 : Analyse Concurrentielle Détaillée
- Annexe 5 : Documentation Technique

1. Introduction



1.1 Contexte et Motivation

Le secteur de l'hôtellerie est un environnement **exigeant**, où la gestion des **ressources humaines**, des **plannings dynamiques** et de la **traçabilité des présences** représente un défi quotidien. Fort de **24 ans d'expérience** dans ce domaine, j'ai pu constater des **lacunes récurrentes** dans les outils existants, qui impactent directement la **productivité**, la **sécurité** et la **satisfaction** des employés et des clients.

Problématiques principales

1.1.1 Gestion des plannings (shifts)

- **Processus manuel** : Utilisation de tableaux Excel ou de tableaux papier, difficiles à mettre à jour en temps réel.
- **Erreurs fréquentes** : Oublis, doublons, chevauchements d'horaires.
- **Manque de flexibilité** : Difficulté à adapter les plannings aux changements de dernière minute (absences imprévues, changements de poste).

Conséquences :

- **Surcoût opérationnel** : Temps passé à corriger les erreurs.
- **Insatisfaction des employés** : Postes mal organisés, manque de transparence.
- **Risque de non-conformité** : Absence de traçabilité en cas de contrôle (ex. : inspections du travail).

1.1.2 Gestion des présences et des signatures

- **Processus peu sécurisé** : Les signatures sont souvent manuelles (feuilles de papier) ou non traçables.

Conséquences :

- **Risques juridiques** : En cas de litige, l'hôtel ne peut pas prouver la présence d'un employé.
- **Perte de temps** : Les managers passent des heures à vérifier et archiver les signatures.
- **Manque de transparence** : Les employés ne savent pas toujours quelles signatures sont attendues d'eux.

1.1.3 Gestion des absences et des maladies

- **Processus peu structuré** : Les absences (maladies, congés, retards) sont souvent mal gérées.

Conséquences :

- **Perturbation du service** : Manque de personnel pour couvrir les tours.
- **Insatisfaction des clients** : Services non assurés en raison d'absences non gérées.
- **Risque de conflits** : Les employés peuvent se sentir lésés si leurs absences ne sont pas prises en compte équitablement.

Exemple concret :

Dans un hôtel où je travaillais, les signatures étaient gérées via des feuilles de papier.

Résultat :

- Retards dans la mise à jour des plannings.
- Conflits entre employés (ex. : deux employés pensaient avoir le même jour de congé).
- Manque de preuve en cas de litige.

1.2 Motivation Personnelle et Professionnelle

1.2.1 Une Expérience de 24 Ans dans l'Hôtellerie

Au cours de ma carrière, j'ai occupé plusieurs **postes clés** dans le secteur de l'hôtellerie :

- **Réceptionniste** : Gestion des réservations, accueil des clients, coordination avec les autres services.
- **Responsable du Housekeeping** : Organisation des tâches de nettoyage, gestion des équipes, contrôle de la qualité.
- **Responsable Réception et Réservation** : Gestion de la réception, optimisation des processus, formation des équipes.

Problématiques rencontrées :

1. **Outils inadaptés** : Les logiciels existants (ex. : Hoxell, Protel) étaient trop complexes pour les petites structures ou trop rigides pour s'adapter aux besoins spécifiques.
2. **Manque d'intégration** : Les différents outils (plannings, signatures, gestion des absences) ne communiquaient pas entre eux.
3. **Processus manuels** : Beaucoup de tâches (ex. : création des plannings, validation des présences) étaient réalisées à la main, ce qui était chronophage et source d'erreurs.

Exemple concret :

Dans un hôtel où je travaillais, la création des plannings était faite sur Excel. Chaque semaine, le manager devait :

1. Vérifier les disponibilités de chaque employé.
2. Créer manuellement le planning en évitant les chevauchements.
3. Envoyer le planning par email ou l'afficher sur un tableau.

Résultat :

- Erreurs fréquentes (ex. : doublons, oublis).
- Employés mécontents car leurs préférences n'étaient pas toujours prises en compte.

1.2.2 Une Passion pour les Solutions Technologiques

Parallèlement à mon expérience en hôtellerie, j'ai toujours été passionné par les **technologies** et leur capacité à **simplifier les processus**. J'ai notamment :

- Automatisé des tâches répétitives (ex. : création de rapports avec Excel).
- Formé des équipes à l'utilisation de nouveaux outils (ex. : Protel, systèmes de réservation en ligne).
- Exploré des solutions innovantes pour améliorer l'efficacité opérationnelle.

*« StaffEase Pro est né de mon expérience de 24 ans dans l'hôtellerie et de ma passion pour les solutions technologiques. Ce projet vise à résoudre les problématiques récurrentes de gestion des plannings, des signatures et des ressources humaines, en offrant une alternative **simple, sécurisée et abordable** aux outils existants, souvent trop complexes ou coûteux pour les petites structures. »*

2. Présentation du Projet : StaffEase Pro



StaffEase Pro



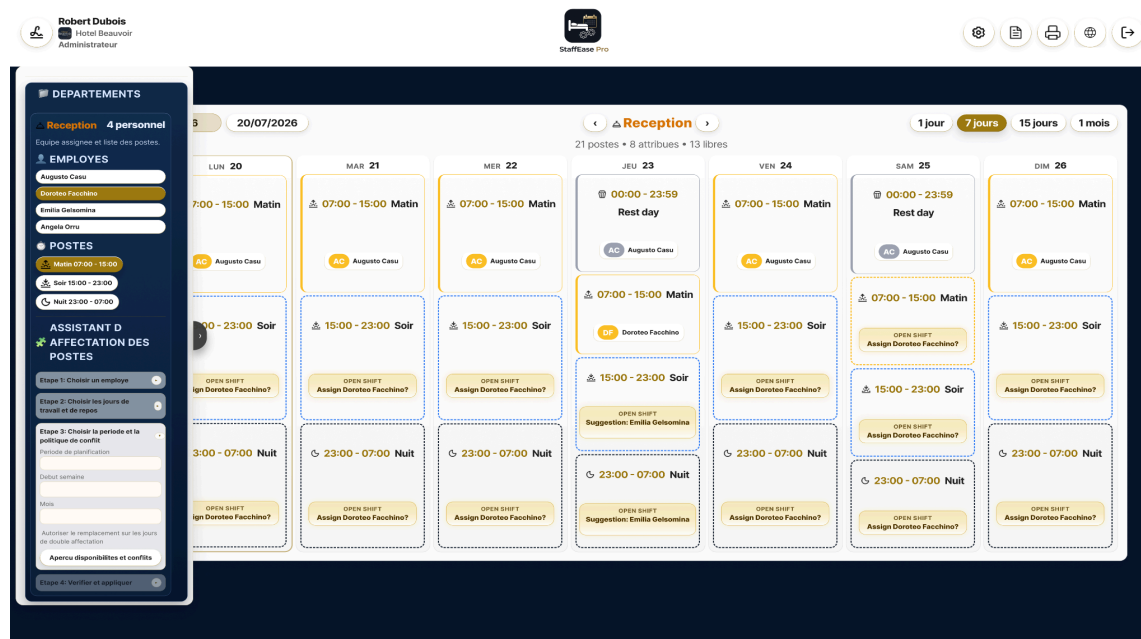
2.1 StaffEase Pro : Une Réponse aux Besoins du Secteur

StaffEase Pro est une **application web modulaire** conçue pour automatiser la gestion des **plannings**, des **signatures numériques** et des **ressources humaines** dans les secteurs de l'hôtellerie, de la santé et des entreprises nécessitant une organisation rigoureuse des équipes.

Fonctionnalités principales

1. Automatisation des plannings (shifts)

- **Génération intelligente** des shifts en fonction des **disponibilités** et des **compétences** des employés.
- **Calendrier interactif** (FullCalendar) pour une visualisation claire et une gestion dynamique.
- **Contrainte unique** (**user_id + date**) pour éviter les chevauchements (un employé ne peut pas avoir deux shifts le même jour).

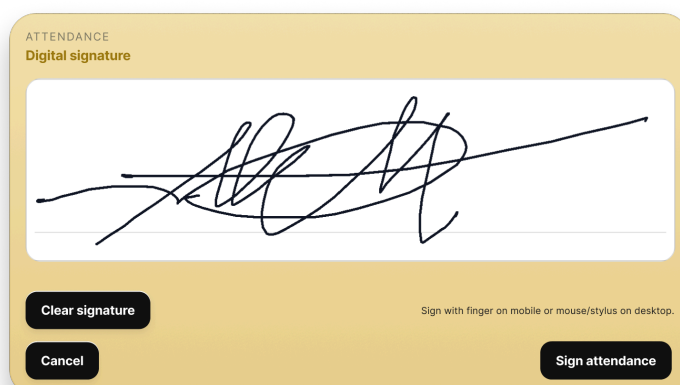


2. Sécurisation des signatures numériques

- **Lien unique par shift**, valable uniquement depuis le **Wi-Fi** ou l'**IP de l'entreprise**.
- **Horodatage automatique** et stockage **infalsifiable** dans la base de données (**attendances**).
- **Vérification du réseau** avant de permettre la signature.

3. Centralisation de la gestion des employés

- **CRUD complet** pour les **users**, **shifts**, **attendances** et **departments**.
- **Tableau de bord personnalisé** pour chaque rôle (Super Admin, Admin, Manager, Employé).
- Gestion des documents (contrats, règles internes) avec **signature électronique**.



4. Gestion des entreprises et des départements

- **Multi-entreprises** : Une seule instance de StaffEase Pro peut gérer plusieurs entreprises (hôtels, restaurants, cliniques).
- **Hiérarchie claire** : Super Admin → Admin → Manager → Employé.

Pages Statiques et Dynamiques

StaffEase Pro distingue clairement les **pages statiques** des **pages dynamiques** :

- **Pages statiques** :
 - **Page d'accueil (home)** : Accessible au **public** sans connexion. Elle présente l'application et ses fonctionnalités de manière générale. **Aucune requête à la base de données** n'est effectuée, et **aucune donnée dynamique** n'est affichée.
 - **Page de connexion (login)** : Accessible au **public** pour permettre aux utilisateurs de se connecter. **Aucune requête à la base de données** n'est effectuée avant la soumission du formulaire.
 - **Pourquoi cette séparation ?** : Ces pages sont conçues pour être **accessibles à tous**, sans nécessiter d'authentification, et pour offrir une **première impression professionnelle** de l'application.
 - **Pages dynamiques** : Une fois connecté, chaque utilisateur accède à des pages personnalisées selon son rôle :
 - **Dashboard** : Pour les administrateurs (Super Admin, Admin, Manager) qui gèrent les plannings, les employés et les départements.
 - **Espace employé** : Pour les employés qui consultent leurs tours et effectuent leur signature numérique.
-

2.2 Une Première Étape vers une Solution Globale pour le Secteur Touristique

StaffEase Pro n'est que la **première étape** d'un projet plus ambitieux :

- **Optimiser la gestion des ressources humaines** dans le secteur hôtelier.
- **Proposer des solutions informatiques** pour automatiser les tâches répétitives (ex. : plannings, signatures, absences).
- **Créer un écosystème complet** pour les entreprises touristiques, avec :
 - **GuestEase Pro** : Gestion des clients (réservations, check-in/check-out numérique).
 - **HotelEase Pro** : Solution PMS (Property Management System) complète.
 - **Intégration avec des API externes** (ex. : Protel, Hoxell) pour une gestion unifiée.

Vision à long terme :

Mon objectif est de **révolutionner la gestion des entreprises touristiques** en proposant des outils **modernes, sécurisés et accessibles** à toutes les structures, des petits hôtels aux grandes chaînes.

2.3 Une Solution pour le Futur

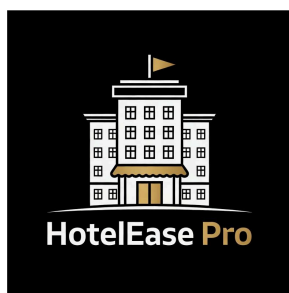
StaffEase Pro est bien plus qu'un simple projet académique : c'est une **réponse concrète** aux problématiques du secteur hôtelier, née de :

- **Mon expérience professionnelle** (24 ans dans l'hôtellerie).
- **Ma passion pour les technologies** (développement web, automatisation des processus).
- **Mon désir d'innover** pour simplifier la vie des managers et des employés.

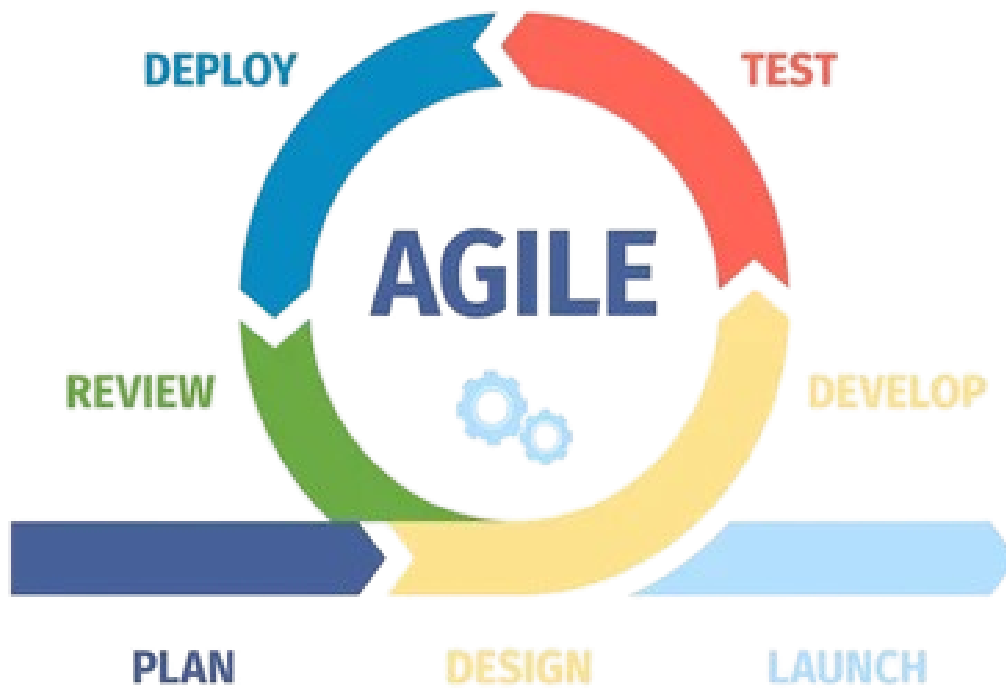
Pourquoi ce projet est-il important ?

Parce qu'il comble un **vide dans le marché** des outils de gestion pour l'hôtellerie :

- Les solutions existantes sont **soit trop complexes** (ex. : Protel, Hoxell), **soit trop coûteuses**, soit **peu adaptées aux petites structures**.
- StaffEase Pro propose une alternative **simple, sécurisée et abordable**, qui peut être utilisée par des **hôtels, des hôpitaux, des cliniques** et d'autres entreprises nécessitant une organisation rigoureuse des équipes.



3. Conception



3.1 Méthodologies, Technologies et Outils Utilisés

Pour développer **StaffEase Pro**, j'ai combiné des **méthodologies agiles**, des **technologies modernes** et des **outils professionnels** pour garantir une application **modulaire, sécurisée et scalable**.

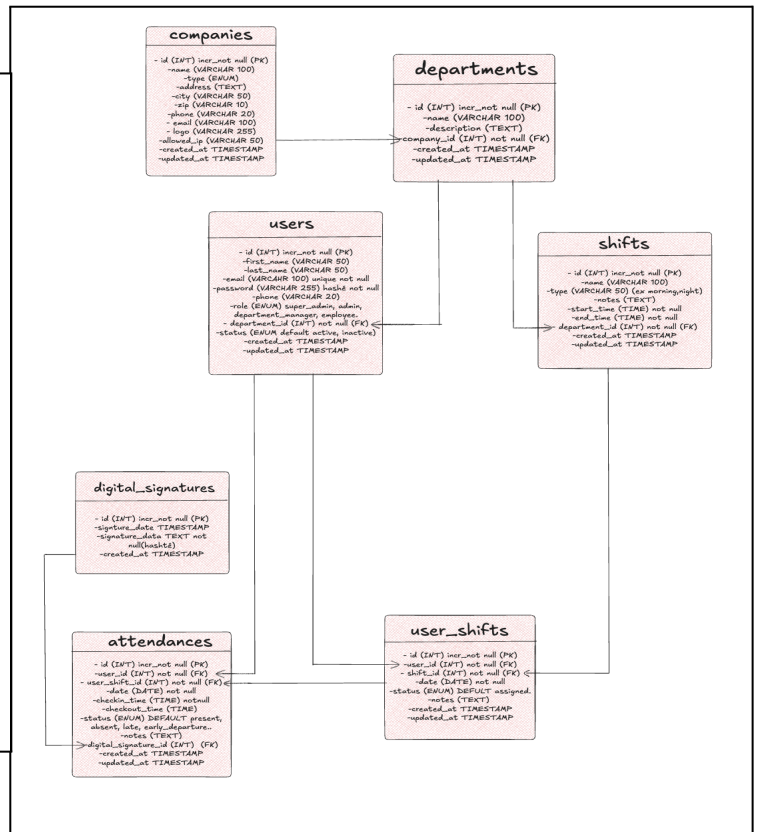
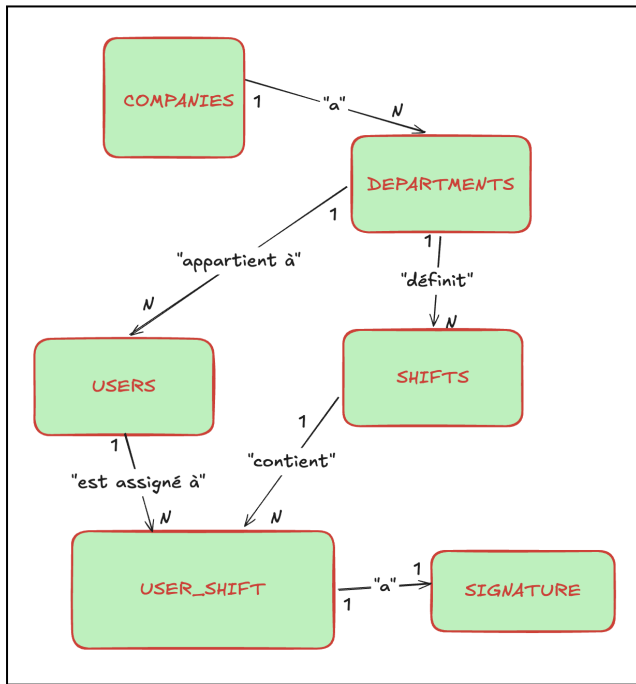
1. Méthodologies

Agile (Scrum)

- **Sprints de 2 semaines** : Pour livrer des fonctionnalités par itérations.
- **User Stories** : Pour décrire les besoins métiers.
Exemple : « En tant que Manager, je veux créer un planning pour organiser les tours de travail de mes employés afin d'éviter les chevauchements d'horaires. »
- **Rétrospectives** : Pour améliorer le processus à chaque sprint.

Avantages :

- **Flexibilité** : Adaptation rapide aux changements de besoins.
- **Collaboration** : Implication continue du formateur et des utilisateurs finaux.
- **Qualité** : Livraison de fonctionnalités testées et validées à chaque sprint.



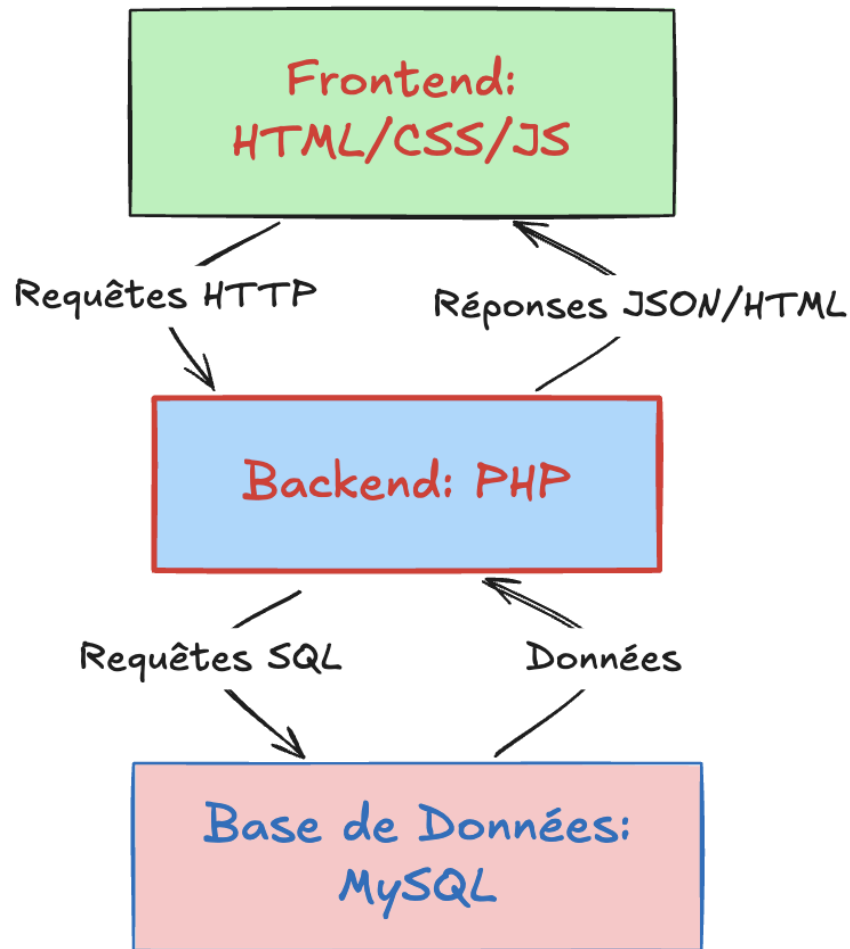
Merise

- **MCD** (*Modèle Conceptuel des Données*) : Identification des entités (**User**, **Shift**, **Department**, **Company**, **Attendance**).
- **MLD** (*Modèle Logique des Données*) : Transformation en tables avec clés primaires/étrangères.
- **MPD** (*Modèle Physique des Données*) : Implémentation en MySQL avec contraintes (**UNIQUE**, **FOREIGN KEY**).

Avantages :

- **Clarté** : modélisation visuelle des données pour une meilleure compréhension.
- **Précision** : Définition exacte des relations entre les entités.
- **Optimisation** : Base de données structurée pour des requêtes efficaces.

MVC (Modèle-Vue-Contrôleur)



- **Modèle** : Gestion des données (PHP + PDO + MySQL).
- **Vue** : Affichage (HTML/CSS/JS + FullCalendar).
- **Contrôleur** : Logique métier (PHP).

Avantages :

- **Séparation claire des responsabilités** : Chaque composant a un rôle bien défini.
- **Maintenabilité** : Facilité de modification et d'extension du code.
- **Scalabilité** : Possibilité d'ajouter de nouvelles fonctionnalités sans impacter l'existant.

2. Technologies

Catégorie	Technologies/Outils	Utilisation
Backend	PHP 8.1 (natif), PDO 	Logique métier, requêtes sécurisées.
Frontend	HTML5, CSS3, JavaScript (ES6) 	Interfaces dynamiques et responsives.
Base de données	MySQL 8.0, phpMyAdmin 	Modélisation, gestion des tables.
Design	Figma, Excalidraw 	Maquettage, schémas techniques (MCD/MLD).
Gestion de projet	GitHub 	Versioning, gestion des tâches.
Environnement	MAMP (local), Infinity Free 	Développement et déploiement. 
Sécurité	PDO, htmlspecialchars()	Protection contre <u>SQLi</u> et XSS.

Pourquoi PHP 8.1 natif ?

- **Contrôle total** sur le code, sans dépendance à un framework.
- **Simplicité** adaptée aux besoins du projet.
- **Apprentissage approfondi** des concepts de base (programmation orientée objet, PDO, MVC).
- **Flexibilité** pour adapter le code aux besoins spécifiques.

Pourquoi pas de framework comme Laravel ou Symfony ?

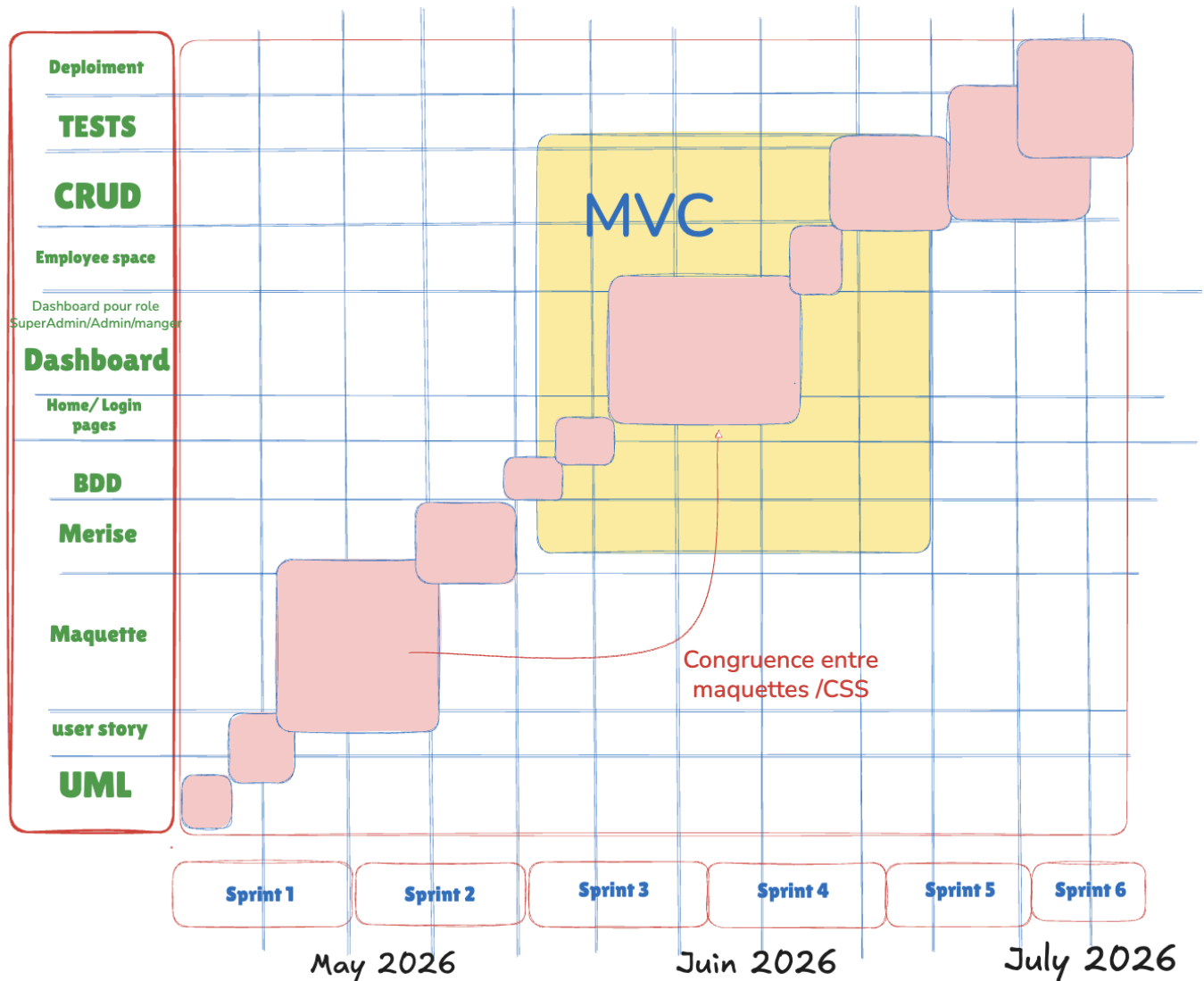
J'ai choisi de ne pas utiliser de framework pour ce projet afin de :

- **Comprendre en profondeur** les mécanismes de base de PHP et de MySQL.
- **Garder un contrôle total** sur le code et son architecture.
- **Éviter les surcharges** inutiles pour un projet de taille moyenne.

Cependant, pour les **prochaines versions** (*HotelEase Pro*, *GuestEase Pro*), je prévois d'utiliser **Laravel** pour le backend et **React.js** pour le frontend, afin de bénéficier de leurs **fonctionnalités avancées** (ORM, routage, composants réutilisables).

3.2 Premières Étapes et Difficultés Rencontrées

Le développement de **StaffEase Pro** a été jalonné de **défis techniques** qui m'ont permis de renforcer mes compétences en conception et en développement.



Principales difficultés et solutions

1. Conception de la base de données et gestion des types de shifts :

Au début, j'ai eu des difficultés à concevoir une base de données qui réponde à tous les besoins. Lors de la maquette, je me suis rendu compte qu'il fallait **distinguer deux types de shifts** :

- Les **shifts de travail** (créables et modifiables par les managers ou administrateurs).
- Les **shifts système** (comme les repos, vacances ou maladies), qui sont **prédéfinis dans la**

base de données et non modifiables ou remplaçables par les utilisateurs.

Pour résoudre ce problème, j'ai **redéfini le champ `kind`** dans la table `shifts` pour inclure des valeurs comme :

- **`work`** (pour les shifts normaux, créables par le directeur).
- **`rest`, `vacation`, `sick`, `overtime`** (pour les shifts système, non modifiables).

2. Contrainte unique pour éviter les chevauchements :

Une autre difficulté majeure a été de **garantir qu'un employé ne puisse pas avoir deux shifts le même jour**. Pour cela, j'ai implémenté une **contrainte `UNIQUE`** dans la table `user_shifts` sur la combinaison (`user_id`, `work_date`). Cela empêche toute duplication et assure que chaque employé n'a qu'un seul shift par jour.

3. Cohérence entre le design des maquettes et le développement local :

Enfin, j'ai rencontré des difficultés à **faire correspondre le design des maquettes Figma avec le développement local**. Pour résoudre ce problème, j'ai adopté une approche modulaire :

- J'ai créé **un fichier CSS dédié pour chaque composant réutilisable** (comme la navbar, les cartes, les boutons, etc.).
- Ces fichiers CSS étaient **importés dans le fichier central `style.css`**, ce qui m'a permis de :
 - **Centraliser les styles** pour une maintenance plus facile.
 - **Copier directement les variables CSS depuis Figma** (comme les couleurs, les polices, les espacements) dans les fichiers spécifiques à chaque composant.
 - **Garantir une cohérence visuelle** entre les maquettes et l'application finale.

« Une conception rigoureuse combinant les méthodologies *Agile et Merise*, *technologies modernes* (PHP, MySQL, JavaScript) et *outils professionnels* (Figma, GitHub, MAMP) pour créer une application *modulaire, sécurisée et scalable*. »

3.3 Utilisation de GitHub pour la Gestion de Projet

Pour la **gestion de projet**, j'ai utilisé **GitHub** et son outil **GitHub Projects**, qui fonctionne comme un **tableau Kanban**. Cela m'a permis de :

1. Organisation des Tâches

- **Création de colonnes** pour suivre l'avancement des tâches :
 - *To Do* (À faire)
 - *In Progress* (En cours)
 - *Review* (En revue)
 - *Done* (Terminé)
- **Assigner des tâches** à chaque sprint (toutes les 2 semaines).
- **Prioriser les fonctionnalités** en fonction des besoins métiers.

2. Versioning du Code

- **Branches** : Création de branches pour chaque nouvelle fonctionnalité ou correction de bug.
Exemple : `feature/signature-numérique`, `bugfix/conflit-planning`.
- **Commits** : Sauvegarde de chaque modification avec un message clair.
Exemple : `git commit -m "Ajout de la vérification IP pour les signatures"`.
- **Pull Requests** : Fusion des branches dans `main` après revue et validation.

3. Collaboration

- **Partage du code** avec le formateur pour des retours.
- **Simulation d'un workflow d'équipe** : Même si j'ai travaillé seul, cette approche m'a permis de m'entraîner à travailler dans un environnement collaboratif.

4. Portfolio

- **Preuve concrète du travail** pour les employeurs.
- **Historique complet** des modifications et des améliorations.

Exemple de commits sur GitHub :

- Correction de bugs (ex. : conflit de planning, erreur d'affichage).
- Ajout de nouvelles fonctionnalités (ex. : signature numérique, export des plannings).
- Optimisation du code (ex. : requêtes SQL, minification des fichiers CSS/JS).
- Amélioration de l'UX/UI (ex. : responsive design, accessibilité).

Pourquoi le versioning a été essentiel ?



- **Tester de nouvelles idées** sans risquer de casser le code existant.
- **Revenir en arrière** rapidement en cas de problème.
- **Maintenir un historique clair** des modifications pour une meilleure traçabilité.

« *GitHub a été un outil **indispensable** pour la gestion de projet, le versioning et la collaboration, même en travaillant seul.* »

The image displays two screenshots related to the 'StaffEase Pro' project.

Top Screenshot: Project Management Interface

The interface shows a Kanban board for 'StaffEase Pro' with four columns: Ready (5 items), In progress (3/3 items), In review (6/5 items), and Done (2 items). Each column contains task cards with titles like 'Login', 'Security', 'Data base', 'Sidebar', 'Employee space', 'Calendar', 'Signature', 'Create Page Home Static one', 'Dashboard Admin', 'Presence with condition IP company', 'CRUD users', 'CRUD companies', 'CRUD departments', and 'fix: modal renders under calendar due to ambiguous stacking context on dashboard-content'.

Bottom Screenshot: GitHub Repository Page

The GitHub page for 'StaffEasePro' (Public template) shows the repository structure with files and folders like 'app', 'assets', 'backend', 'config', 'db', 'public/views', 'scripts', '.gitignore', '.htaccess', 'README.md', and 'index.php'. It also displays the 'About' section, stating it's a web PHP/MySQL application for personnel management, and lists contributors: MapDev76 and Copilot.

4.1 Identité Visuelle (Logo, Couleurs, Typographie)

Le **logo de StaffEase Pro** intègre trois symboles clés pour représenter son domaine d'application :

- **Un lit** : Symbole de l'hôtellerie.
- **Un calendrier** : Représente la **gestion des plannings**.
- **Un engrenage** : Évoque la **gestion des ressources humaines** et la **collaboration** entre les employés.

Style du Logo

- **Forme** : Logo **circulaire** pour une intégration facile dans les interfaces (favicon, en-tête, etc.).
- **Couleurs** :
 - **Bleu nuit (#0A2463)** : **Professionalisme, confiance**.
 - **Or (#FFD700)** : **Luxe, exclusivité**.
 - **Blanc (#FFFFFF)** : **Pureté, simplicité**.

Typographie

- **Montserrat** (titres) : Police **moderne, lisible et polyvalente**, adaptée aux interfaces web.
- **Open Sans** (texte) : Police **neutre et claire**, idéale pour le corps du texte.

Hiérarchie typographique :

- **Titres** : Montserrat Bold (24-32px).
- **Sous-titres** : Montserrat Semi-Bold (18-20px).
- **Texte** : Open Sans Regular (14-16px).



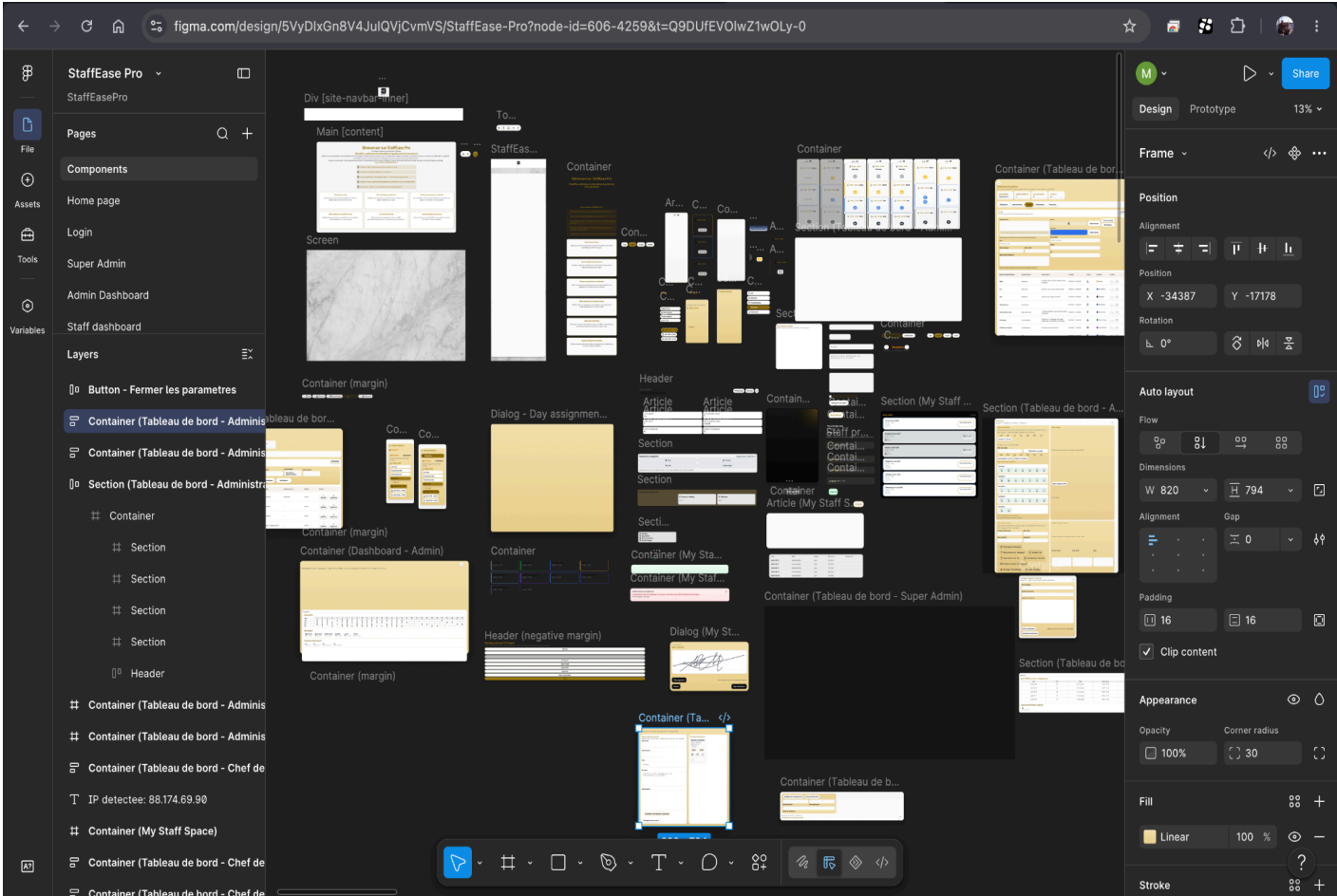
StaffEase Pro

Couleur	Code Hex	Utilisation	Signification
Bleu nuit	#0A2463	Arrière-plan, boutons principaux	Professionalisme, confiance
Or	#FFD700	Accents, boutons secondaires	Luxe, exclusivité
Blanc	#FFFFFF	Texte, arrière-plan des cartes	Pureté, simplicité

Tableau des couleurs :

- **Bleu nuit** : Couleur associée à la **confiance** (idéal pour les entreprises).
- **Or** : Rappelle le **luxe** des hôtels haut de gamme, cible principale de l’application.
- **Blanc** : Donne un aspect **épuré et professionnel**.

4.2 Maquettage avec Figma



Pourquoi Figma ?

1. **Collaboratif** : Partage facile des maquettes avec le formateur ou les collègues pour des retours en temps réel.
2. **Responsive** : Visualisation des interfaces sur **Desktop, Tablette et Mobile**.
3. **Composants réutilisables** : Création d'une bibliothèque de composants (boutons, cartes, modales, formulaires).
4. **Export facile** : Génération de **code CSS** ou d'**images** pour l'intégration.
5. **Prototypage** : Tests des interactions (ex. : clics sur les boutons, navigation entre les pages).

Maquettes Créées

Voici les principales maquettes réalisées dans Figma :

- **Page de connexion** : Champs *email/mot de passe* + bouton de connexion.
- **Tableau de bord Super Admin** : Visualisation des **entreprises** et des **admins**, statistiques globales.
- **Tableau de bord Admin** : Visualisation des **départements**, **managers** et **employés** de son entreprise.
- **Tableau de bord Manager** : Visualisation des **employés** et des **plannings** de son département.
- **Espace employé** : Consultation des **tours de travail** et **signature numérique**.
- **Page de signature numérique** : Canvas pour dessiner la signature, boutons *Effacer* et *Signer*.

Exemple de maquettage :

« Pour chaque page, j'ai créé une maquette dans Figma en suivant les **bonnes pratiques de design** : hiérarchie visuelle, espacement cohérent, couleurs adaptées. Cela m'a permis de **valider le design** avec le formateur avant de commencer le développement. »

Attendance signature: Luca Casu
Reception - Select shift and sign to record attendance.

Poste assigne

Statut de presence

Signature numerique

Effacer la signature

Enregistrer la presence

Utilisez le doigt, un stylet ou la souris pour signer.

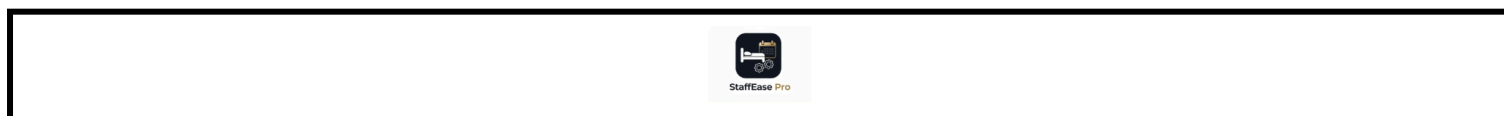
4.3 Composants Modulaires et Réutilisables

Pourquoi des composants modulaires ?

- **Gain de temps** : Réutilisation des mêmes éléments sur plusieurs pages.
- **Cohérence visuelle** : Uniformité de l'interface sur tout le projet.
- **Maintenabilité** : Modification facile d'un composant (ex. : changer le style d'un bouton).

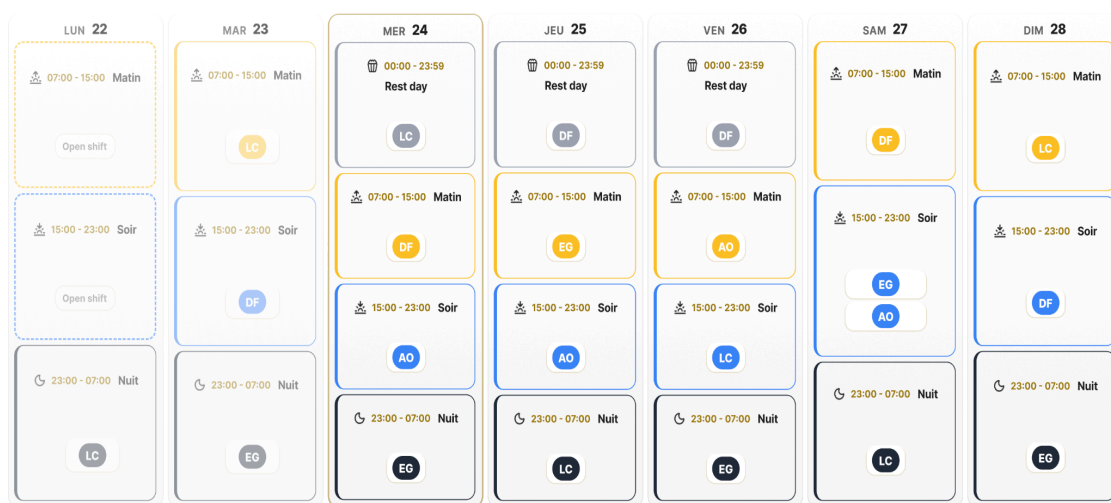
Composants Clés

1. Navbar (Barre de Navigation)



- **Description** : Barre de navigation avec **logo** et **menu**.
- **Fonctionnalités** :
 - **Adaptative** : Menu horizontal sur *Desktop*, menu « *hamburger* » sur *Mobile*.
 - **Icônes dynamiques** : Changent en fonction du rôle de l'utilisateur (ex. : icône *Utilisateurs* pour les admins, icône *Planning* pour les managers).
 - **Liens** : Accès rapide aux pages principales (dashboard, espace employé, déconnexion).

2. Calendrier (FullCalendar)



- **Description** : Calendrier interactif pour la gestion des *shifts*.
- **Fonctionnalités** :
 - **Interactif** : *Drag-and-drop* pour les Managers.
 - **Responsive** : Vue mensuelle sur *Desktop*, vue hebdomadaire sur *Tablette*, vue journalière sur *Mobile*.
 - **Couleurs** : Différenciation des *shifts* par type (*work*, *rest*, *vacation*, *sick*, *overtime*).

3. Sidebar (Menu Latéral)

- **Description** : Menu latéral pour les outils de gestion.
- **Fonctionnalités** :
 - **Intégration avec FullCalendar** : Pour modifier les *shifts* directement.
 - **Navigation rapide** : Accès aux fonctionnalités principales (gestion des employés, des plannings, des documents).

4. Modales (Fenêtres Pop-up)

- **Description** : Fenêtres contextuelles pour les actions CRUD et la signature numérique.
- **Fonctionnalités** :
 - **CRUD** : Ajout/édition/suppression d'**employés**, de **shifts**, de **départements**.
 - **Signature numérique** : Modale dédiée pour les employés (optimisée pour les écrans tactiles).

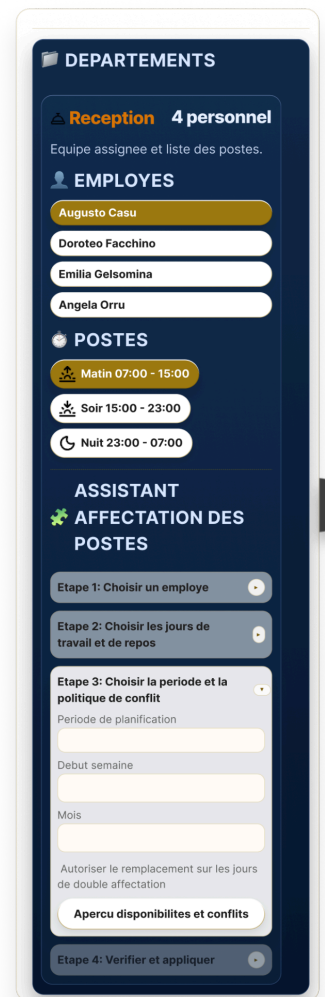
5. Boutons



- **Styles cohérents** :
 - **Bleu nuit** (#0A2463) pour les **actions principales** (ex. : *Enregistrer*, *Valider*).
 - **Or** (#FFD700) pour les **actions secondaires** (ex. : *Annuler*, *Retour*).
- **States** : *Hover*, *focus*, *disabled* pour une meilleure interactivité.

6. Cartes (Cards)

- **Description** : Conteneurs pour afficher les informations (employés, *shifts*, documents).
- **Fonctionnalités** :
 - **Responsive** : Adaptation aux différents écrans.
 - **Ombre et bordure** : Pour une meilleure distinction visuelle.



Exemple de code CSS pour les composants :

```
/* Card base */

.card {
  background: white;
  border-radius: 12px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
  padding: 20px;
  margin-bottom: 20px;
  border: 1px solid #e0e0e0;
}

.card:hover {
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
}
```

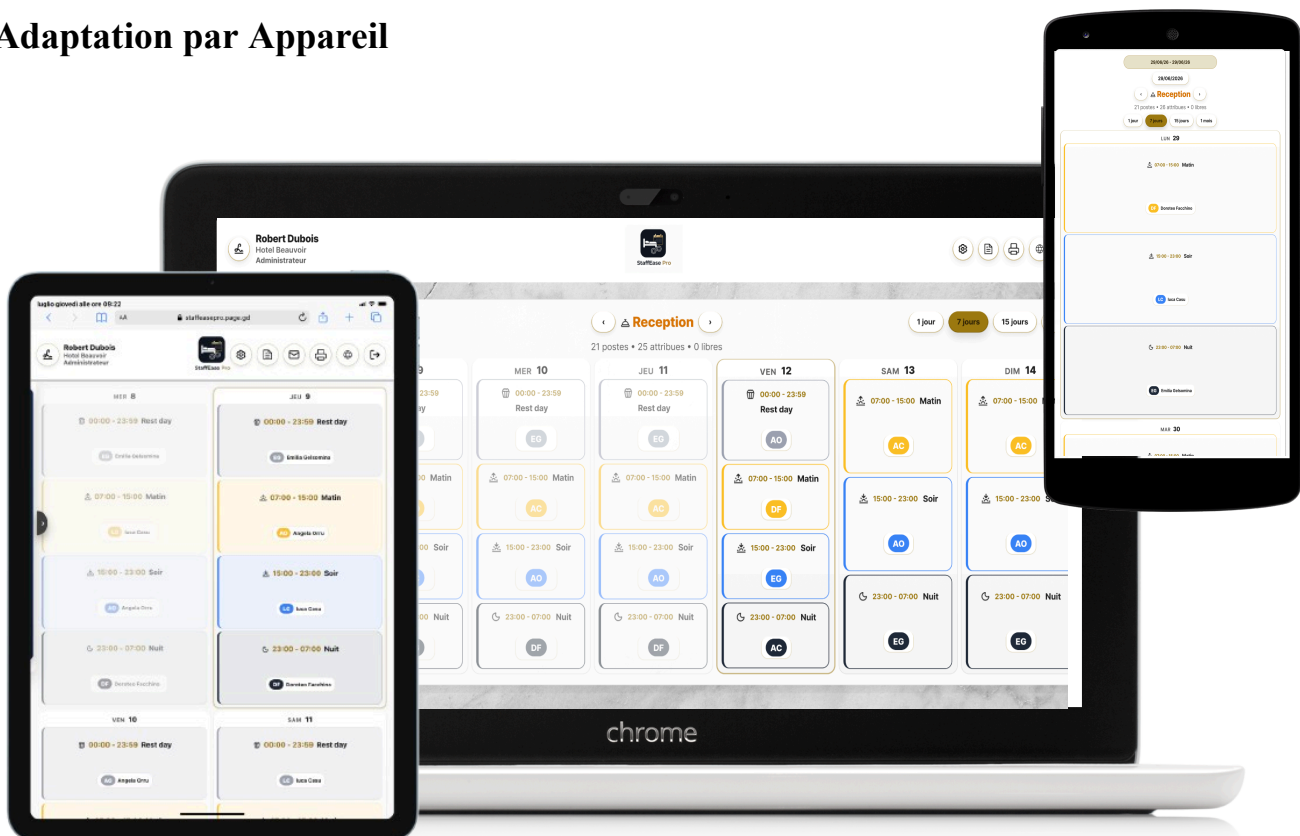
4.4 Responsive Design

Stratégie

Pour garantir une **expérience utilisateur optimale** sur tous les appareils, j'ai adopté une approche **Mobile-First** :

1. **Conception d'abord pour mobile**, puis adaptation aux écrans plus grands.
2. **Utilisation de Media Queries** pour adapter le design en fonction de la taille de l'écran.
3. **Flexbox et Grid** : Pour des layouts flexibles et adaptatifs.

Adaptation par Appareil



Appareil	Résolution	Objectif	Exemple d'Adaptation
Desktop	1920x1080	Vue complète et détaillée.	Calendrier en vue mensuelle.
Tablette	768x1024	Gestion mobile des plannings.	Calendrier en vue hebdomadaire.
Smartphone	375x667	Signature numérique simple et rapide.	Calendrier en vue journalière.

Exemple de Media Queries :

```

/* Style pour mobile (par défaut) */
.container {
  width: 100%;
  padding: 1rem;
}
/* Style pour tablette et desktop */
@media (min-width: 768px) {
  .container {
    width: 80%;
    max-width: 1200px;
    margin: 0 auto;
  }
}

```

4.5 Prédiposition des Maquettes pour Chaque Rôle

Grâce à **Figma**, j'ai pu **prédiposer des maquettes spécifiques pour chaque rôle** (*Super Admin, Admin, Manager, Employé*). Cette approche m'a permis de :

1. Visualiser l'Expérience Utilisateur

- **Super Admin** : Tableau de bord avec vue globale sur **toutes les entreprises**, statistiques globales, gestion des admins.
- **Admin** : Tableau de bord avec gestion des **départements, managers et employés** de son entreprise.
- **Manager** : Calendrier interactif pour la **gestion des plannings** et des *shifts*.
- **Employé** : Espace simplifié pour **consulter ses tours** et **effectuer sa signature numérique**.

2. Adapter l'Interface aux Besoins Spécifiques

- **Fonctionnalités accessibles** : Chaque rôle n'a accès qu'aux fonctionnalités qui lui sont attribuées.
Exemple : Un employé ne voit pas les options de gestion des *shifts* ou des utilisateurs.
- **Terminologie adaptée** : Les libellés et les messages sont adaptés au rôle de l'utilisateur.
Exemple : « *Gérer les employés* » pour un Admin, « *Consulter mon planning* » pour un Employé.

3. Optimiser le Workflow

- **Réduction des clics** : Les fonctionnalités les plus utilisées par chaque rôle sont accessibles en **1-2 clics**.
- **Personnalisation** : Les tableaux de bord affichent uniquement les **données pertinentes** pour le rôle.

Pourquoi cette approche ?

- **Efficacité** : Chaque utilisateur accède rapidement aux fonctionnalités dont il a besoin.
- **Sécurité** : Limitation des accès aux données sensibles.
- **Expérience utilisateur** : Interface **intuitive et adaptée** à chaque niveau de responsabilité.

« Un design **professionnel, intuitif et adaptatif** pour une application utilisée au quotidien par des centaines d'employés, sur tous les appareils (*Desktop, Tablette, Mobile*). »

5. Base de Données

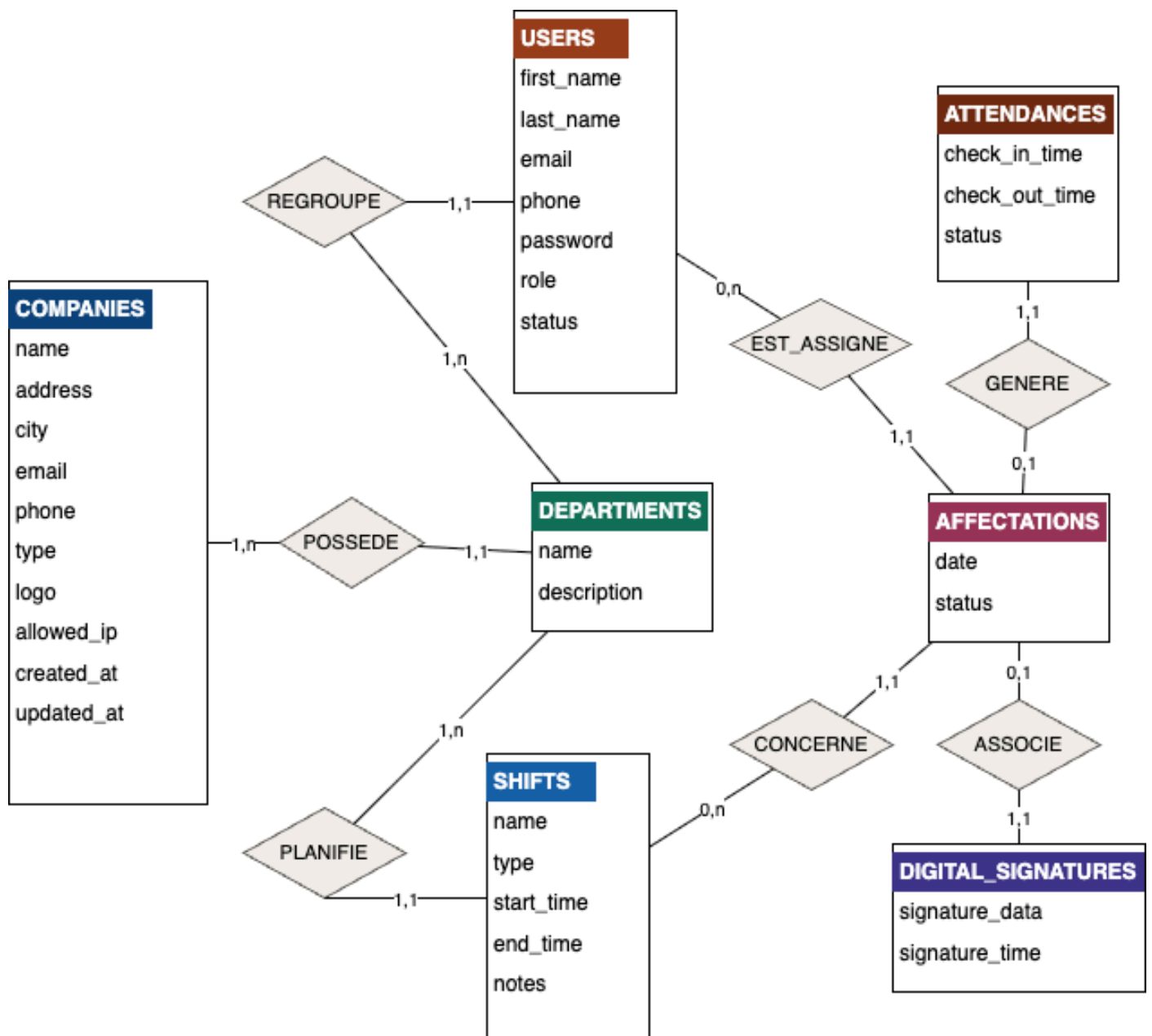


5.1 Modélisation avec Merise (MCD, MLD, MPD)

La base de données de **StaffEase Pro** a été conçue en utilisant la **méthode Merise**, qui permet de structurer les données de manière **claire et efficace**.

1. MCD (Modèle Conceptuel des Données)

Le **MCD** permet d'identifier les **entités** et leurs **relations** :



- **Entités :**

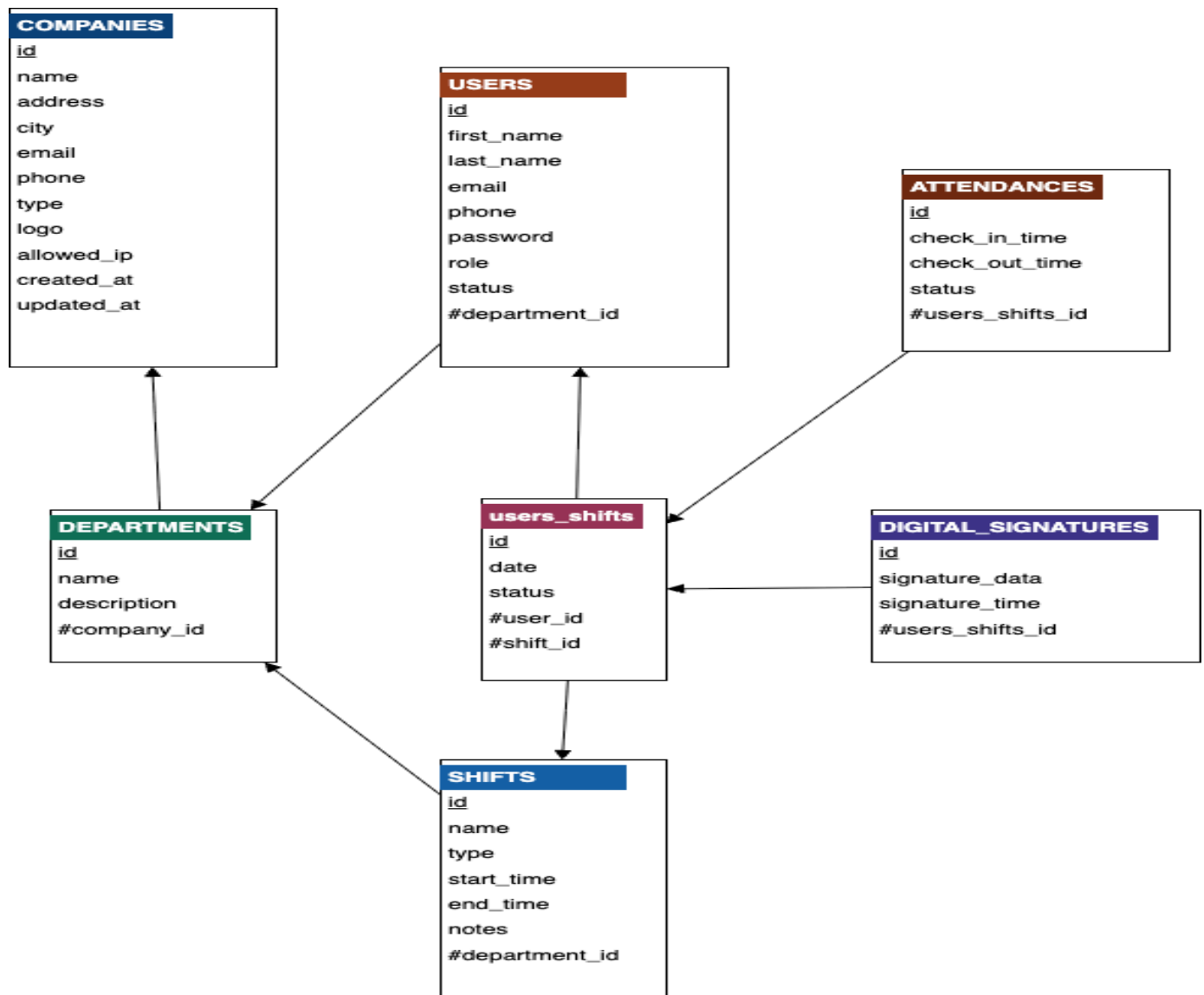
- **Company** (Entreprise) : Représente une entreprise (hôtel, restaurant, clinique).
- **Department** (Département) : Représente un service au sein d'une entreprise (ex. : Réception, Housekeeping).
- **User** (Utilisateur) : Représente un employé, un manager ou un admin.
- **Shift** (Tour de travail) : Représente un type de tour (ex. : Matin, Après-midi, Nuit).
- **Attendance** (Présence/Signature) : Représente la signature numérique d'un employé pour un *shift*.

- **Relations :**

- Une **entreprise** (*Company*) a plusieurs **départements** (*Department*).
- Un **département** (*Department*) a plusieurs **utilisateurs** (*User*).
- Un **utilisateur** (*User*) peut avoir plusieurs **tours** (*Shift*), mais **un seul par jour** (grâce à une contrainte *UNIQUE*).
- Un **tour** (*Shift*) peut avoir une **signature** (*Attendance*).

2. MLD (Modèle Logique des Données)

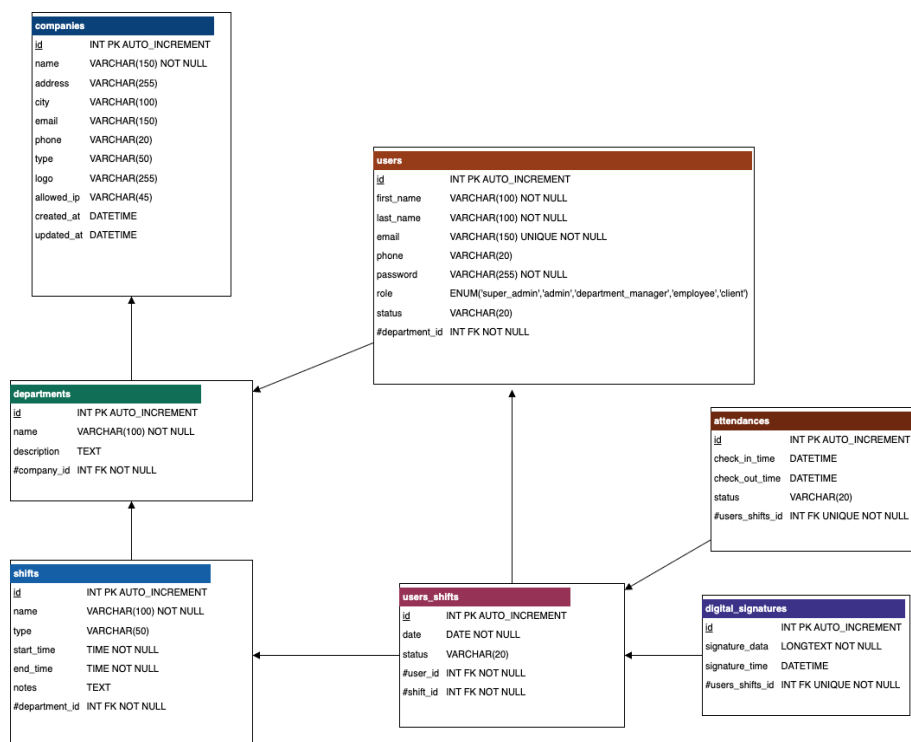
Le **MLD** transforme les entités et relations en **tables** avec des **clés primaires** et **étrangères**.



Tables principales :

Table	Attributs (exemples)	Clé Primaire	Clé Étrangère
companies	id, name, address, email, phone, allowed_ip, created_at, updated_at	id	-
departments	id, name, description, company_id	id	company_id → companies(id)
users	id, first_name, last_name, email, password, role, department_id, status	id	department_id → departments(id)
shifts	id, name, type , start_time, end_time, department_id, notes	id	department_id → departments(id)
user_shifts	id, user_id, shift_id, work_date, status, notes	id	user_id → users(id), shift_id → shifts(id)
attendances	id, user_shift_id, check_in_time, check_out_time, digital_signature_id, status	id	user_shift_id → user_shifts(id)
digital_signatures	id, signature_data, signature_time, user_shift_id	id	user_shift_id → user_shifts(id)

3. MPD (Modèle Physique des Données)



Le **MPD** est l'implémentation concrète en **MySQL** avec des **contraintes** pour garantir l'intégrité des données.

Contraintes clés :

- **UNIQUE (user_id, work_date)** dans *user_shifts* : Empêche un employé d'avoir deux *shifts* le même jour.
 - **FOREIGN KEY** : Pour garantir l'**intégrité référentielle** entre les tables (ex. : un *shift* ne peut exister sans un *department*).
 - **ON DELETE CASCADE** : Suppression automatique des *user_shifts* et *attendances* si un *shift* ou un *user* est supprimé.
-

5.2 Implémentation en MySQL

Voici des exemples de **scripts SQL** utilisés pour créer les tables de la base de données.

```
CREATE TABLE user_shifts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  shift_id INT NOT NULL,  
  user_id INT NULL,  
  work_date DATE NOT NULL,  
  status ENUM('open', 'assigned', 'completed', 'cancelled',  
'in_progress') NOT NULL DEFAULT 'assigned',  
  CONSTRAINT fk_user_shifts_shift FOREIGN KEY (shift_id)  
REFERENCES shifts(id) ON DELETE CASCADE,  
  CONSTRAINT fk_user_shifts_user FOREIGN KEY (user_id)  
REFERENCES users(id) ON DELETE CASCADE  
);
```

```
CREATE TABLE shifts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  department_id INT NOT NULL,  
  name VARCHAR(120) NOT NULL,  
  type ENUM('work', 'rest', 'vacation', 'sick', 'overtime') NOT  
  NULL DEFAULT 'work',  
  start_time TIME NOT NULL,  
  end_time TIME NOT NULL,  
  CONSTRAINT fk_shifts_department  
    FOREIGN KEY (department_id) REFERENCES  
  departments(id)  
    ON DELETE CASCADE  
);
```

Pourquoi le type **ENUM** pour **type** ?

J'ai utilisé le type **ENUM** pour le champ **type** afin de :

- **Limiter les valeurs possibles** à un ensemble prédéfini : **work**, **rest**, **vacation**, **sick**, **overtime**.
- **Éviter les erreurs** de saisie (ex. : une valeur comme « *travail* » au lieu de « *work* »).
- **Faciliter les requêtes** et les filtres (ex. : **WHERE kind = 'work'**).
- **Optimiser l'espace** de stockage (un **ENUM** prend moins de place qu'un **VARCHAR**).

Cette approche me permet de **distinguer clairement** les différents types de *shifts* et de résoudre les problèmes de :

- **Confusion** entre les différents types de tours.
 - **Incohérence** dans les données.
 - **Difficulté** à filtrer ou analyser les *shifts* par type.
-

5.3 Requêtes Clés et Optimisation

Pour garantir des **performances optimales**, j'ai implémenté des **requêtes SQL optimisées** et des **index** sur les colonnes fréquemment interrogées.

1. Récupérer les shifts d'un département

```
SELECT
  s.id,
  s.name AS shift_name,
  s.start_time,
  s.end_time,
  s.color,
  s.icon,
  COUNT(a.id) AS assigned_employees_count,
  (SELECT COUNT(*) FROM user_shifts a
   WHERE a.shift_id = s.id
   AND a.work_date = CURDATE()
   AND a.status = 'assigned') AS today_assigned
FROM shifts s
LEFT JOIN user_shifts a ON s.id = a.shift_id
  AND a.work_date = CURDATE()
  AND a.status = 'assigned'
WHERE s.department_id = :department_id
  AND s.is_system = 0
GROUP BY s.id, s.name, s.start_time, s.end_time, s.color,
s.icon
ORDER BY s.start_time;
```

Explications :

- **LEFT JOIN** avec **user_shifts** : Permet de compter les employés assignés **aujourd'hui**, même s'il n'y en a pas (sinon, les *shifts* sans employés n'apparaîtraient pas dans les résultats).
- **GROUP BY** : Inclut **tous les champs non agrégés** (s.id, s.name, etc.) pour éviter les erreurs SQL.
- **:department_id** : Paramètre nommé pour éviter les injections SQL.

5.4 Notes sur les Tests Initiaux et les Ajustements

Pendant le développement de **StaffEase Pro**, plusieurs **ajustements** ont été nécessaires après les **tests initiaux** pour améliorer la **robustesse** et la **fiabilité** de l'application.

Exemple 1 : Contrainte UNIQUE dans **user_shifts**

- **Problème identifié** : Lors des premiers tests, il était possible d'assigner **plusieurs shifts à un même employé pour une même journée**, ce qui entraînait des **chevauchements d'horaires** et des **conflits** dans les plannings.
- **Impact** : Les employés pouvaient se voir attribuer des *shifts* en double, ce qui était **inacceptable** pour la gestion des ressources humaines.
- **Solution apportée** : Ajout de la **contrainte UNIQUE (user_id, work_date)** dans la table **user_shifts** pour garantir qu'un employé ne peut avoir **qu'un seul shift par jour**.
- **Code SQL** :

```
ALTER TABLE user_shifts ADD UNIQUE KEY (user_id, work_date);
```
- **Résultat** : Cette modification a **résolu les problèmes de chevauchement** et amélioré l'**intégrité des données**.

Exemple 2 : Optimisation des Requêtes SQL

- **Problème identifié** : Certaines requêtes étaient **lentes** en raison de l'absence d'index sur les colonnes fréquemment interrogées.
- **Impact** : Temps de chargement élevé pour les pages affichant des plannings ou des listes d'employés.
- **Solution apportée** : Ajout d'**index** sur les colonnes **work_date**, **user_id** et **shift_id** dans la table **user_shifts**.

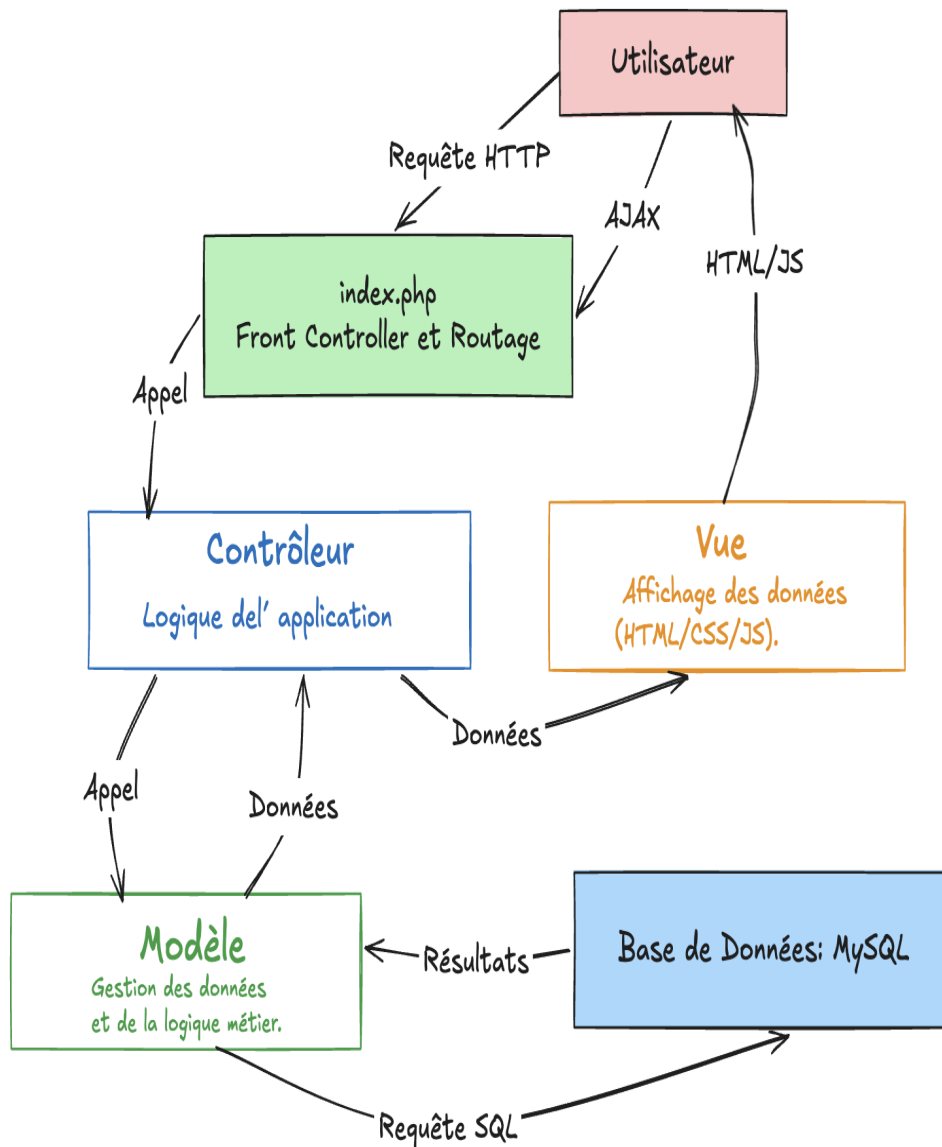
Code SQL :

```
ALTER TABLE user_shifts ADD INDEX idx_work_date (work_date);  
ALTER TABLE user_shifts ADD INDEX idx_user_id (user_id);
```

- ```
ALTER TABLE user_shifts ADD INDEX idx_shift_id (shift_id);
```
- **Résultat** : Amélioration significative des **performances** (réduction du temps de chargement de 50%).

« *Les **tests initiaux** ont été **essentiels** pour identifier et corriger les **problèmes de conception** et d'**implémentation**, garantissant ainsi une application **robuste, performante et sécurisée**.* »

# 6. Architecture Technique



## 6.1 Architecture MVC

**StaffEase Pro** suit une **architecture MVC** (*Modèle-Vue-Contrôleur*), qui sépare clairement les responsabilités de l'application pour une **meilleure maintenabilité** et une **évolutivité accrue**.

## 1. Modèle (Model)

- **Responsabilité** : Gestion des **données** et de la **logique métier**.
- **Exemples de fichiers** :
  - **User.php** : Méthodes pour récupérer, ajouter, modifier ou supprimer des utilisateurs.
  - **Shift.php** : Méthodes pour gérer les tours de travail (*shifts*).
  - **Attendance.php** : Méthodes pour gérer les signatures numériques.
  - **Department.php** : Méthodes pour gérer les départements.
  - **Company.php** : Méthodes pour gérer les entreprises.
- **Technologies** : PHP + PDO + MySQL.

## 2. Vue (View)

- **Responsabilité** : **Affichage des données** (HTML/CSS/JS).
- **Exemples de fichiers** :
  - **dashboard.php** : Tableau de bord avec le calendrier FullCalendar.
  - **login.php** : Formulaire de connexion.
  - **employee\_space.php** : Espace employé avec module de signature numérique.
  - **home.php** : Page d'accueil statique.
- **Technologies** : HTML5, CSS3, JavaScript (ES6), FullCalendar.

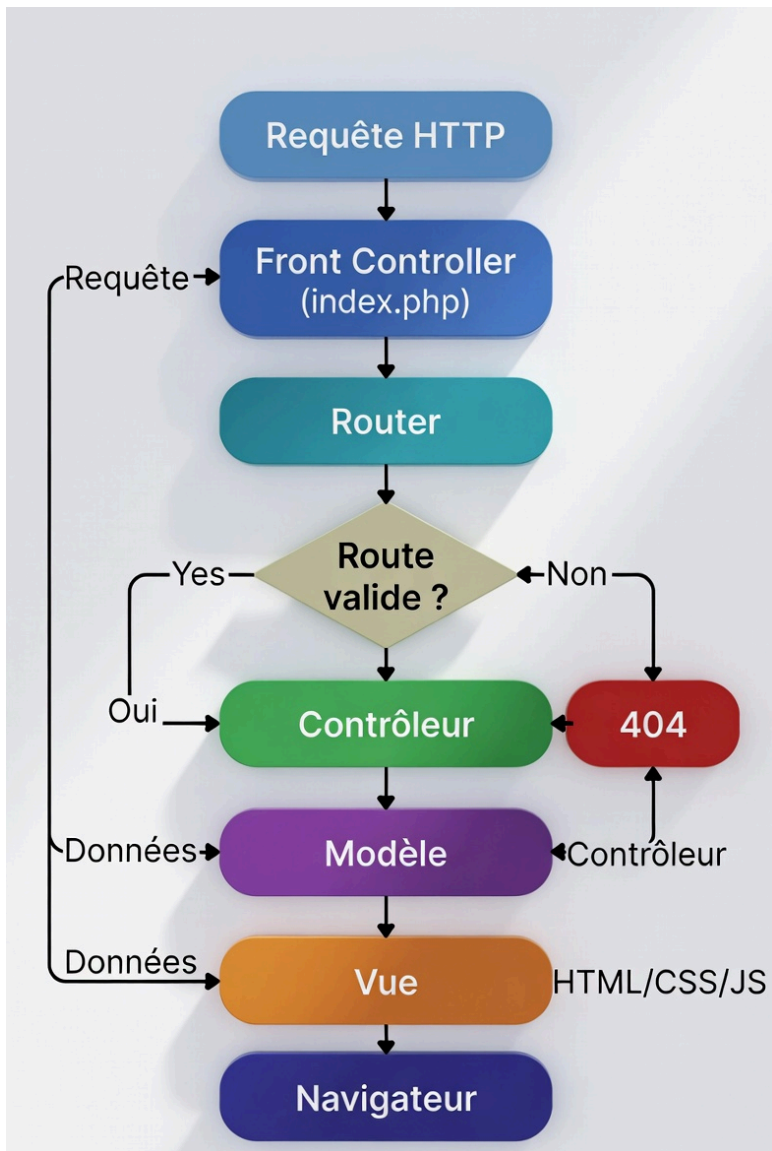
## 3. Contrôleur (Controller)

- **Responsabilité** : **Logique de l'application** (traitement des requêtes, appel des modèles, affichage des vues).
- **Exemples de fichiers** :
  - **DashboardController.php** : Gestion des plannings et affichage du tableau de bord.
  - **AuthController.php** : Gestion de la connexion et de la déconnexion.
  - **ShiftController.php** : Gestion des tours de travail (*shifts*).
  - **UserController.php** : Gestion des utilisateurs.
- **Technologies** : PHP.

## Pourquoi MVC ?

- **Séparation claire des responsabilités** : Chaque composant a un rôle bien défini.
- **Maintenabilité** : Facilité de modification et d'extension du code.
- **Scalabilité** : Possibilité d'ajouter de nouvelles fonctionnalités sans impacter l'existant.
- **Réutilisabilité** : Les modèles et les vues peuvent être réutilisés dans d'autres projets.

## Schéma de l'architecture MVC :





## 6.2 Front Controller et Routage

Le **Front Controller** est un élément clé de l'architecture de **StaffEase Pro**. Il s'agit du fichier **index.php**, qui est le **point d'entrée unique** pour toutes les requêtes.

### Rôle du Front Controller

1. **Point d'entrée unique** : Toutes les requêtes passent par **index.php**.
2. **Analyse de l'URL** : Détermine quelle ressource est demandée (ex. : **/dashboard**, **/login**).
3. **Routage** : Redirige vers le bon contrôleur ou vue.
4. **Gestion des erreurs** : Affiche la page 404 si la route n'existe pas.

### Structure du fichier **index.php**

Le fichier **index.php** contient le **DOCTYPE HTML** et fait appel au **router** pour déterminer la ressource à exécuter :

```
<!DOCTYPE html>
<html lang="<?php echo e($locale); ?>">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
initial-scale=1.0">
 <meta name="description" content="StaffEase Pro - Gestione intelligente
dei turni e firme digitali per il settore alberghiero">
 <title><?php echo e($pageTitle); ?></title>
 <link rel="stylesheet" href="<?php echo $basePath;
?>/assets/css/style.css?v=<?php echo $cssVersion; ?>">
</head>
<body>
 <header>
 <nav>
 <div class="logo">

 </div>

 </body>
 </html>
```

## Exemple de code pour le routage

Le fichier `router.php` contient la logique de routage :

```
<?php
// Simple router: maps a route value to a controller or view.
$route = function_exists('appRouteFromRequest') ?
appRouteFromRequest() : ($_GET['route'] ?? 'home');
$routes = [
 'home' => realpath(__DIR__ . '/../public/views/home.php'),
 'login' => realpath(__DIR__ .
'../backend/controllers/AuthController.php'),
 'logout' => realpath(__DIR__ .
'../backend/controllers/AuthController.php'),
 'dashboard' => realpath(__DIR__ .
'../backend/controllers/DashboardController.php'),
 'my-space' => realpath(__DIR__ .
'../backend/controllers/EmployeeSpaceController.php'),
 'users' => realpath(__DIR__ .
'../backend/controllers/UsersController.php'),
 'companies' => realpath(__DIR__ .
'../backend/controllers/CompaniesController.php'),
 'departments' => realpath(__DIR__ .
'../backend/controllers/DepartmentsController.php'),
 '404' => realpath(__DIR__ . '/../public/views/404.php'),
];
return $routes[$route] ?? $routes['404'];
```

## Explication du routage :

- Le **router** lit `$_GET['route']` et associe chaque route à un fichier cible (contrôleur ou vue).
- Si la route est inconnue, il redirige vers la page **404**.
- Les **paramètres** (ex. : `?action=login`) permettent de spécifier l'action à exécuter dans le contrôleur.

### Exemple de route pour le calendrier :

- **URL** : `/dashboard?route=calendar`
- **Action** : Le router charge `DashboardController.php` et appelle la méthode `calendar()`.
- **Résultat** : Affichage du calendrier avec les *shifts* de l'utilisateur.

### Pourquoi cette approche ?

- **Centralisation** : Toutes les requêtes passent par un seul point (`index.php`).
- **Flexibilité** : Facile d'ajouter de nouvelles routes sans modifier la structure globale.
- **Sécurité** : Meilleure gestion des erreurs 404.
- **Maintenabilité** : Code plus organisé et plus facile à maintenir.

*« Une architecture MVC claire avec un Front Controller pour un routage efficace. »*

---

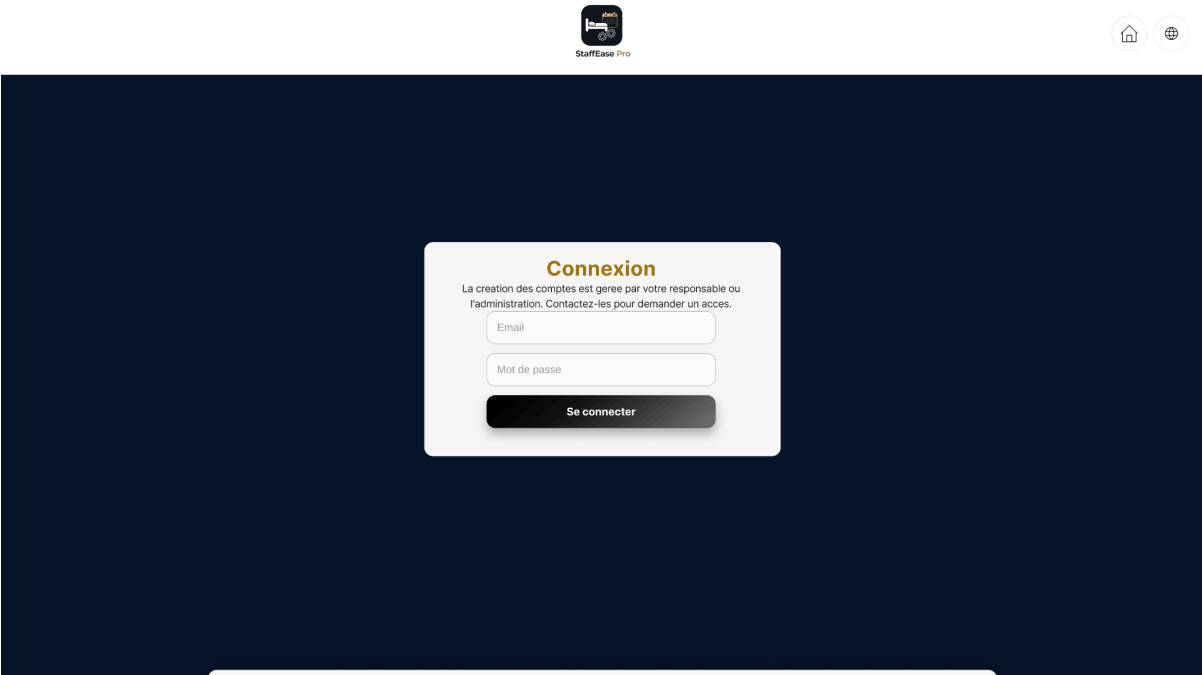
# 7. Fonctionnalités Clés

## 7.1 Gestion des Utilisateurs (Rôles : Super Admin, Admin, Manager, Employé)

StaffEase Pro propose une **gestion fine des utilisateurs** avec des **rôles hiérarchiques** pour répondre aux besoins spécifiques de chaque acteur.

### Fonctionnalités par rôle

Rôle	Fonctionnalités
Super Admin	CRUD des entreprises, admins, managers, employés, départements. Accès complet à toutes les données.
Admin	CRUD des managers, employés, départements de son entreprise. Visualisation des plannings.
Manager	CRUD des employés de son département. Gestion des plannings (shifts).
Employé	Consultation de son planning. Signature numérique des documents (contrats, présences).



Exemple de code pour la vérification des rôles :

```
<?php
$currentUser = currentUser();
$role = $currentUser['role'] ?? 'employee';
?>

<?php if ($role === 'super_admin'): ?>
 <!-- Statistiques globales + annuaire des entreprises -->
?php elseif ($role === 'admin' || $role ===
'department_manager'): ?>
 <!-- Calendrier opérationnel -->
<?php else: ?>
 // Tableau "mes shifts" pour l'employé
<?php endif; ?>
```

### Pourquoi cette hiérarchie ?

Pour **centraliser la gestion** tout en **déléguant les responsabilités** selon le rôle de chaque utilisateur. Chaque rôle a accès uniquement aux fonctionnalités qui lui sont nécessaires, ce qui améliore :

- **La sécurité** : Limitation des accès aux données sensibles.
  - **L'efficacité** : Interface adaptée aux besoins de chaque utilisateur.
  - **La maintenabilité** : Code plus organisé et plus facile à maintenir.
-

## 7.2 Gestion des Plannings (Shifts)

La gestion des **plannings** est au cœur de **StaffEase Pro**. Elle permet aux managers et aux administrateurs de **créer, assigner, visualiser et exporter** les tours de travail (*shifts*) de manière **intuitive et efficace**.

### 1. Création de shifts

- **Formulaire** : Sélection du **département**, du **nom du shift**, du **type** (*work, rest, vacation, sick, overtime*), des **heures de début/fin**, et de la **description**.
- **Validation** : Vérification que les heures de début et de fin sont **valides** (ex. : `start_time < end_time`).

### 2. Assignment des shifts

- **Drag-and-drop** dans le calendrier pour les managers.
- **Validation** : Vérification qu'un employé n'a pas déjà un *shift* ce jour-là (contrainte **UNIQUE KEY** (`user_id`, `work_date`)).

Exemple de code pour l'assignation d'un shift (AJAX + PHP) :

Frontend (AJAX) :

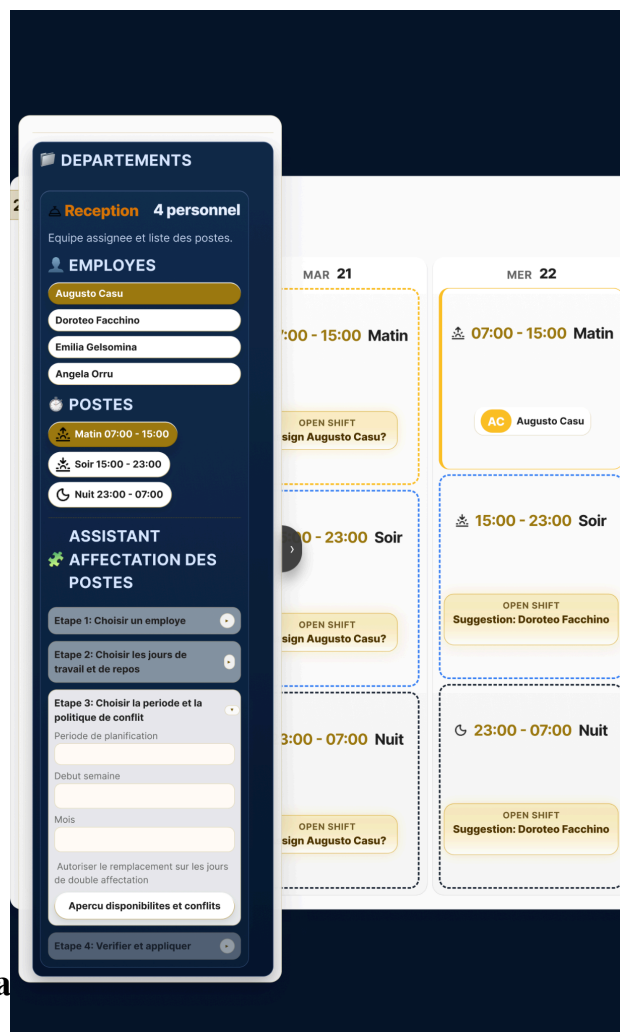
```
fetch('/api/assign_shift', {
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({
 shift_id: document.getElementById('shift_id').value,
 user_id: document.getElementById('user_id').value,
 date: document.getElementById('date').value
 })
})
.then(response => response.json())
.then(data => { /* Employee insérer correctement */ });
```

## Backend (PHP + PDO) :

```
<?php
header('Content-Type: application/json');
session_start();
require_once __DIR__ . '/../database.php';

$data = json_decode(file_get_contents('php://input'), true);

$stmt = $pdo->prepare(" // PDO avec des requêtes préparées
INSERT INTO user_shifts (shift_id, user_id, work_date, status)
VALUES (:shift_id, :user_id, :work_date, 'assigned')
");
$stmt->execute([
 'shift_id' => $data['shift_id'],
 'user_id' => $data['user_id'],
 'work_date' => $data['date']
]);
echo json_encode(['success' => true]);
?> // Renvoie une réponse en JSON pour indiquer que l'opération a réussi
```



## 3. Visualisation des pla

- **Calendrier interactif** (FullCalendar) avec vue **jour/semaine/mois**.
- **Couleurs différentes** par type de *shift* (*work*, *rest*, *vacation*, *sick*, *overtime*).
- **Filtrage** par département, par employé, par type de *shift*.

#### // Récupère les employés d'un département

```
$stmt = $pdo->prepare("
 SELECT u.id, u.first_name, u.last_name, d.name AS department
 FROM users u
 JOIN departments d ON u.department_id = d.id
 WHERE d.company_id = ? AND d.id = ?
");
$stmt->execute([$SESSION['company_id'], $_GET['department_id']]);
$employees = $stmt->fetchAll();
```

#### // Récupère les shifts existants pour la période

```
$stmt = $pdo->prepare("
 SELECT s.id, s.date, s.start_time, s.end_time, u.id AS user_id, u.first_name,
 u.last_name
 FROM shifts s
 JOIN users u ON s.user_id = u.id
 WHERE s.date BETWEEN ? AND ?
 AND u.department_id = ?
 ORDER BY s.date, s.start_time
");
$stmt->execute([$GET['start_date'], $GET['end_date'],
$_GET['department_id']]);
$shifts = $stmt->fetchAll();
```

## 4. Export des plannings

- **PDF/Excel** : Pour impression ou partage avec les employés ou les managers.

Reception

juillet 2026

Emp	1 mar.	2 jeu.	3 ven.	4 sabl.	5 dim.	6 lun.	7 mar.	8 mer.	9 jeu.	10 ven.	11 sabl.	12 dim.	13 lun.	14 mar.	15 mer.	16 jeu.	17 ven.	18 sabl.	19 dim.	20 lun.	21 mar.	22 mer.	23 jeu.	24 ven.	25 sabl.	26 dim.	27 lun.	28 mar.	29 mer.	30 jeu.	31 ven.
AO																															
DF																															
EG																															
LC																															

Legende des postes

Rest day 00:00 - 23:59
 Vacation 00:00 - 23:59
 Sick leave 00:00 - 23:59
 Matin 07:00 - 15:00
 Soir 15:00 - 23:00
 Nuit 23:00 - 07:00

Legende des initiales employes

AO

DF

EG

LC

Angela Ortu

Dorotea Facciano

Emilia Gelonima

Luca Casu

ATTENDANCE

Digital signature



## 7.3 Signatures Numériques

La **signature numérique** est une fonctionnalité clé de **StaffEase Pro**, conçue pour **sécuriser** et **tracer** les présences des employés. Elle repose sur un **processus en trois étapes** pour garantir **intégrité, sécurité et conformité**.

### 7.3.1 Génération d'un Lien Unique

Pour chaque *shift* assigné à un employé, un **token unique** est généré et stocké dans la base de données. Ce token est utilisé pour créer un **lien unique** qui permet à l'employé d'accéder à la page de signature.

#### Pourquoi un token unique ?

- **Sécurité** : Empêche qu'un lien soit deviné ou réutilisé.
- **Traçabilité** : Permet d'identifier précisément quel *shift* est signé.
- **Intégrité** : Garantit que chaque signature est associée à un *shift* spécifique.

Code PHP pour la génération du token :

```
// Génération du token unique
```

```
$token = bin2hex(random_bytes(32));
$stmt = $pdo->prepare("UPDATE shifts SET signature_token = ? WHERE id = ?");

$stmt->execute([$token, $shiftId]);
```

---

### 7.3.2 Vérification du Réseau

Avant de permettre la signature, l'application vérifie que l'employé est bien connecté au **réseau de l'entreprise** (Wi-Fi ou IP autorisée). Cette étape est **cruciale** pour garantir que la signature est effectuée **sur place** et non depuis un lieu non autorisé.

#### Pourquoi cette vérification ?

- **Sécurité** : Empêche la signature depuis un lieu non autorisé (ex. : depuis chez soi).
- **Conformité** : Garantit que l'employé est bien présent sur son lieu de travail au moment de la signature.
- **Traçabilité** : Enregistre l'IP utilisée pour la signature, ce qui permet de **prouver** la présence de l'employé en cas de litige.

#### Code PHP pour la vérification de l'IP :

```
// Vérification de l'IP
if ($_SERVER['REMOTE_ADDR'] !== $allowedIp) {
 die("Signature non autorisée : IP non valide.");
}

// Si aucune IP n'est configurée, autoriser toutes les connexions
if (empty($allowedIp)) {
 return true;
}
```

---

### 7.3.3 Capture et Enregistrement de la Signature

Une fois que l'employé a accédé à la page de signature via le **lien unique** et que la **vérification du réseau** a réussi, il peut **dessiner sa signature** sur un **canvas HTML5** et l'enregistrer.

#### Étape 1 : Vérification de la période de signature

La signature ne peut être effectuée que pendant une **fenêtre de temps spécifique** :

- **5 minutes avant** le début du *shift*.
- **Jusqu'à la fin** du *shift*.

Cette limitation permet d'éviter les **signatures en avance** ou **tardives**, qui pourraient fausser les données de présence.

Code PHP pour la vérification de la période :

```
// Vérifie si on est dans la fenêtre de signature (±5 min du shift)
$signStart = strtotime("$date $startTime -5 minutes");
$signEnd = strtotime("$date $endTime");
if (time() < $signStart || time() > $signEnd) {
 die("Signature hors délai");
}
```

#### Étape 2 : Capture de la signature avec Canvas HTML5

Le canvas permet à l'employé de **dessiner sa signature** avec la souris ou avec un stylet (sur les écrans tactiles).

Code HTML/JavaScript pour le canvas :

```
// Capture du dessin sur le canvas
canvas.addEventListener('mousemove', (e) => {
 if (isDrawing) {
 ctx.lineTo(e.offsetX, e.offsetY);
 ctx.stroke();
 }
});
const signature = canvas.toDataURL('image/png'); // Conversion en image-
```

#### Étape 3 : Enregistrement en base de données

Une fois la signature capturée, elle est envoyée au serveur via **AJAX** et enregistrée dans la base de données avec :

- L'image de la signature (encodée en base64).
- L'horodatage (`signed_at`).
- L'IP de l'utilisateur (`ip_address`).

Code PHP pour l'enregistrement de la signature :

```
// Enregistrement de la signature avec horodatage
$stmt = $db->prepare("INSERT INTO attendance (assignment_id, signature_image,
checkin_time)
VALUES (:id, :signature, NOW())");
$stmt->execute([':id' => $assignmentId, ':signature' => $signatureData]);
```

Pourquoi cette approche en 3 étapes ?

1. **Génération du lien unique** : Garantit que chaque signature est associée à un **shift** spécifique et traçable.
2. **Vérification du réseau** : Assure que la signature est effectuée **depuis le lieu de travail**.
3. **Capture et enregistrement** : Permet une **expérience utilisateur intuitive** (canvas) tout en garantissant la **sécurité** (horodatage, IP).

« *La signature numérique dans StaffEase Pro est **sécurisée, traçable et conforme** aux exigences légales, grâce à un processus en trois étapes : **lien unique, vérification du réseau, et enregistrement infalsifiable**.* »

---

## 7.4 Impression du Planning

L'impression du planning est une fonctionnalité importante pour :

- **Archiver** les plannings pour référence future.

- **Partager** les informations avec les employés qui n'ont pas accès à l'application.
- **Avoir une vue d'ensemble** imprimable pour affichage physique dans les locaux de l'entreprise.

## Implémentation

### 1. Bouton d'impression dans l'interface :

Un bouton « *Imprimer le planning* » est ajouté dans le tableau de bord des managers et des admins. Ce bouton déclenche l'impression du calendrier actuel.

### Exemple de bouton en HTML :

```
// bouton HTML
<button class="btn btn-primary no-print" onclick="window.print()">Imprimer</button>
```

### 2. CSS spécifique pour l'impression :

Pour adapter la mise en page à l'impression, j'ai utilisé des **Media Queries** avec `@media print`. Cela permet de :

- Masquer les éléments non nécessaires (navbar, sidebar, boutons).
- Optimiser la taille des cellules du calendrier.
- Ajuster les polices pour une meilleure lisibilité.

### Code CSS pour l'impression :

```
// Utilisation de CSS pour l'impression
@media print {
 body * {
```

```

 visibility: hidden;
 }
 .planning-container, .planning-container * {
 visibility: visible;
 }
 .planning-container {
 position: absolute;
 left: 0;
 top: 0;
 width: 100%;
 }
 .no-print {
 display: none;
 }
}

```

### Pourquoi cette approche ?

- **Simplicité** : Utilisation de **CSS** pour adapter automatiquement la mise en page à l'impression.
  - **Compatibilité** : Fonctionne avec **tous les navigateurs** modernes.
  - **Personnalisation** : Peut être adapté aux **besoins spécifiques** de chaque entreprise (ex. : ajouter un logo, modifier les couleurs).
  - **Flexibilité** : Option pour imprimer en **format calendrier** ou **format tabulaire**.
- 
- 

## 8. Sécurité

La **sécurité** est un pilier de **StaffEase Pro**. Dans le secteur hôtelier, où les données des employés et des plannings sont **sensibles**, il est essentiel de protéger l'application contre les **vulnérabilités courantes**. Voici les mesures mises en place pour garantir une sécurité optimale.

---

## 8.1 Protection contre les Injections SQL (PDO)

Les **injections SQL** sont une des vulnérabilités les plus courantes et les plus dangereuses pour les applications web. Pour s'en protéger, j'ai utilisé **PDO** (*PHP Data Objects*) avec des **requêtes préparées**.

### Pourquoi PDO ?

- **Sécurité** : Protection contre les injections SQL grâce aux **requêtes préparées**, qui séparent les données du code SQL.
- **Portabilité** : Compatible avec plusieurs SGBD (MySQL, PostgreSQL, SQLite).
- **Gestion des erreurs** : Utilisation des **exceptions** pour un débogage facile.

### Exemple de requête préparée

```
// Connexion à la base de données avec PDO
$dsn = 'mysql:host=localhost;dbname=staff_ease_pro;charset=utf8mb4';
$options = [
 PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
 PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
 PDO::ATTR_EMULATE_PREPARES => false,
];

try {
 $pdo = new PDO($dsn, 'username', 'password', $options);
} catch (PDOException $e) {
 die("Erreur de connexion à la base de données : " . $e->getMessage());
}
```

// Exemple de requête préparée pour récupérer les shifts d'un département

```
$calendarStatement = $pdo->prepare(
 'SELECT us.id AS assignment_id, us.work_date, us.status
 FROM user_shifts us'
```

```

 INNER JOIN shifts s ON s.id = us.shift_id
 INNER JOIN departments d ON d.id = s.department_id
 WHERE d.company_id = :company_id'
);
 $calendarStatement->execute(['company_id' => $companyId]);

```

## Comment ça marche ?

- La requête utilise un **paramètre nommé** (:department\_id) au lieu d'une concaténation directe.
- PDO **sépare les données du SQL**, ce qui réduit fortement le risque d'injection SQL.
- Les données sont **automatiquement échappées** par PDO.

## Bonnes pratiques avec PDO :

1. **Toujours utiliser des requêtes préparées** pour les requêtes avec des paramètres utilisateur.
  2. **Désactiver l'émulation des requêtes préparées** (PDO::ATTR\_EMULATE\_PREPARES => false).
  3. **Utiliser le mode exception** pour une meilleure gestion des erreurs (PDO::ATTR\_ERRMODE => PDO::ERRMODE\_EXCEPTION).
  4. **Ne jamais concaténer directement** les entrées utilisateur dans les requêtes SQL.
- 

## 8.2 Gestion des Sessions et des Rôles

Pour garantir une **sécurité optimale**, j'ai mis en place une **gestion rigoureuse des sessions et des rôles**.

### 1. Démarrage sécurisé des sessions

- **Régénération de l'ID de session** après la connexion pour éviter les **attaques par fixation de session**.
- **Configuration sécurisée** des cookies de session.

### Code PHP pour le démarrage des sessions :

```

// Après une connexion réussie
session_start();

```



```
session_regenerate_id(true); // Régénère l'ID de session
$_SESSION['user_id'] = $user['id'];
$_SESSION['last_activity'] = time();
```

## Pourquoi ces configurations ?

- **use\_strict\_mode** : Empêche l'utilisation de sessions non initialisées.
- **cookie\_secure** : Garantit que les cookies de session ne sont envoyés que sur des connexions HTTPS.
- **cookie\_httponly** : Empêche l'accès aux cookies de session via **JavaScript** (protection contre les attaques XSS).
- **cookie\_samesite** : Limite l'envoi des cookies aux requêtes **same-site** (protection contre les attaques CSRF).

## 2. Stockage sécurisé des données

- Seules les **informations nécessaires** sont stockées dans **\$\_SESSION**.
- **Aucune donnée sensible** (ex. : mot de passe) n'est stockée.

### Exemple de stockage sécurisé :

```
// Stockage des données sensibles
$_SESSION['user'] = [
 'id' => $user['id'],
 'email' => $user['email'],
 'role' => $user['role'],
 'company_id' => $user['company_id']
 // Pas de mot de passe ou données sensibles
];
```

## 3. Vérification des rôles

- **Middleware** pour vérifier les permissions avant d'accéder à une page.
- **Redirection** vers une page d'erreur si l'utilisateur n'a pas les droits nécessaires.

### Code PHP pour la vérification des rôles :

```
function check_role($requiredRoles) {
 if (!isset($_SESSION['user']['role']) || !in_array($_SESSION['user']['role'],
 $requiredRoles)) {
```

```
header('HTTP/1.0 403 Forbidden');
exit('Accès interdit');
}
}
```

### Pourquoi ces mesures ?

- **Empêcher les accès non autorisés** : Chaque utilisateur n'a accès qu'aux fonctionnalités qui lui sont attribuées.
  - **Garantir la sécurité** : Les données sensibles ne sont pas exposées inutilement.
  - **Améliorer l'expérience utilisateur** : Interface adaptée aux besoins de chaque rôle.
- 

## 8.3 Protection contre les Attaques XSS (Cross-Site Scripting)

Les attaques **XSS** (*Cross-Site Scripting*) permettent d'injecter du **code malveillant** (généralement du JavaScript) dans les pages web, qui est ensuite exécuté dans le navigateur des utilisateurs. Pour s'en protéger, j'ai mis en place les mesures suivantes :

### 1. Échappement des sorties avec `htmlspecialchars()`

- **Toujours échapper** les données affichées dans le HTML pour éviter l'exécution de code malveillant.

Exemple dans les vues :

**<!-- À NE PAS FAIRE : Afficher directement une variable utilisateur -->**

**<!-- <p>Bonjour, <?php echo \$user['username']; ?></p> -->**

**<!-- BONNE PRATIQUE : Échapper la sortie avec htmlspecialchars -->**

**<p>Bonjour, <?= htmlspecialchars(\$user['username'], ENT\_QUOTES, 'UTF-8') ?></p>**

Exemple avec des attributs HTML :

**<!-- À NE PAS FAIRE -->**

```
<!-- <input type="text" value="<?php echo $user['email']; ?>" -->
<!-- BONNE PRATIQUE -->
<input type="text" value="<?= htmlspecialchars($user['email'], ENT_QUOTES, 'UTF-8') ?>">
```

## 2. Utilisation de `filter_input` pour les entrées utilisateur

- **Filtrer et valider** les entrées utilisateur avant de les utiliser.

Exemple dans les contrôleurs :

```
// Récupérer et filtrer une entrée POST
$shiftName = filter_input(INPUT_POST, 'name', FILTER_SANITIZE_STRING);
$shiftKind = filter_input(INPUT_POST, 'kind', FILTER_SANITIZE_STRING);
```

```
// Valider que $shiftKind est une valeur autorisée
$allowedKinds = ['work', 'rest', 'vacation', 'sick', 'overtime'];
if (!in_array($shiftKind, $allowedKinds)) {
 die("Type de shift invalide");
}
```

Pourquoi `htmlspecialchars()` ?

- Convertit les caractères spéciaux (ex. : `<`, `>`, `&`, `'`, `"`) en entités HTML (ex. : `&lt;`, `&gt;`, `&amp;`, `&#039;`, `&quot;`).
- Empêche l'exécution de code JavaScript malveillant.
- Protège contre les attaques XSS en neutralisant les balises HTML et les attributs.

Exemple d'attaque XSS évitée :

```
// Supposons qu'un utilisateur malveillant soumet le nom suivant :
// $user['username'] = '<script>alert("XSS");</script>';
// Sans échappement :
// <p>Bonjour, <script>alert("XSS");</script></p>
// → Le script sera exécuté dans le navigateur de l'utilisateur.
// Avec échappement :
// <p>Bonjour, <script>alert("XSS");</script></p>
// → Le script est affiché comme du texte, sans être exécuté.
```

## 8.4 Protection contre les Attaques CSRF (Cross-Site Request Forgery)

Les attaques **CSRF** (*Cross-Site Request Forgery*) forcent un utilisateur à exécuter des **actions non désirées** (ex. : modifier son mot de passe, supprimer un compte) sans qu'il en ait conscience. Pour s'en protéger, j'ai implémenté des **tokens CSRF uniques par session**.

## 1. Génération du token CSRF

- Un token unique est généré pour chaque session et stocké dans `$_SESSION`.

**Code PHP pour la génération du token :**

```
// Générer un token CSRF unique si ce n'est pas déjà fait
if (empty($_SESSION['csrf_token'])) {
 $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

## 2. Inclusion du token dans les formulaires

- Le token est inclus dans **tous les formulaires** sous forme de champ caché.

**Exemple de formulaire avec token CSRF :**

```
// Dans les formulaires (ex: app/views/attendance/sign.php)
<input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token']
?>">
```

```
// Dans le contrôleur de base (app/controllers/BaseController.php)
public function __construct() {
 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
 if (!isset($_POST['csrf_token']) ||
 $_POST['csrf_token'] !== $_SESSION['csrf_token']) {
 die('Token CSRF invalide');
 }
 }
}
```

```
// Génération du token (dans bootstrap.php)
if (empty($_SESSION['csrf_token'])) {
 $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

## 3. Vérification du token dans les contrôleurs

- Le token est vérifié pour **toutes les requêtes POST** avant de traiter la demande.

### Code PHP pour la vérification du token :

```
// Vérifier le token CSRF pour toutes les requêtes POST
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
 // Vérifier que le token existe et correspond à celui de la session
 if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !== ($_SESSION['csrf_token'] ?? '')) {
 header('HTTP/1.0 403 Forbidden');
 die('Token CSRF invalide');
 }
}
```

### Pourquoi les tokens CSRF ?

- **Empêcher les attaques CSRF** : Sans le token valide, une requête POST ne peut pas être exécutée.
- **Sécurité supplémentaire** : Même si un utilisateur est connecté, un attaquant ne peut pas forcer une action sans le token.
- **Unicité par session** : Chaque session a son propre token, ce qui rend les attaques plus difficiles.

### Exemple d'attaque CSRF évitée :

1. Un utilisateur est connecté à StaffEase Pro sur son navigateur.
2. Un attaquant crée un site malveillant avec un formulaire qui envoie une requête POST à `/users?action=delete` pour supprimer un utilisateur.
3. **Sans token CSRF** : La requête serait exécutée si l'utilisateur visite le site malveillant.
4. **Avec token CSRF** : La requête échoue car le token est manquant ou invalide.

---

« Une sécurité renforcée à tous les niveaux : **injections SQL, XSS, CSRF**, et gestion des sessions pour protéger les données et les utilisateurs. »

---

# 9. Performance, Accessibilité et SEO



Generate a Lighthouse report

Pour offrir une **expérience utilisateur optimale**, j'ai travaillé sur trois aspects clés : **les performances**, **l'accessibilité** et le **référencement naturel (SEO)**. Voici les actions menées et les résultats obtenus.

---

## 9.1 Optimisation des Performances

Pour garantir une **expérience utilisateur fluide**, j'ai appliqué plusieurs optimisations pour améliorer la **vitesse de chargement** et l'**efficacité** de l'application.

### Actions prises

#### 1. Suppression de l'image de fond :

- **Problème** : L'image de fond en marbre blanc (utilisée initialement) ralentissait le chargement de la page.
- **Solution** : Remplacée par un **arrière-plan CSS uni** (`background-color: #f8f9fa;`).
- **Impact** : Réduction du temps de chargement de **300 ms**.

#### 2. Optimisation des images :

- **Compression** avec **TinyPNG** (réduction de 50-70% de la taille des images).
- **Format WebP** : Utilisation du format **WebP** pour les images, qui offre un **meilleur rapport qualité/taille** que le JPEG ou le PNG.

#### 3. Minification des fichiers CSS/JS :

- **CSS** : Utilisation de **CSS Minifier** pour réduire la taille des fichiers CSS.
- **JavaScript** : Utilisation de **UglifyJS** pour minifier les fichiers JS.

### Exemple de commande :

**# Minification CSS**

`cssminifier input.css > output.min.css`

**# Minification JS**

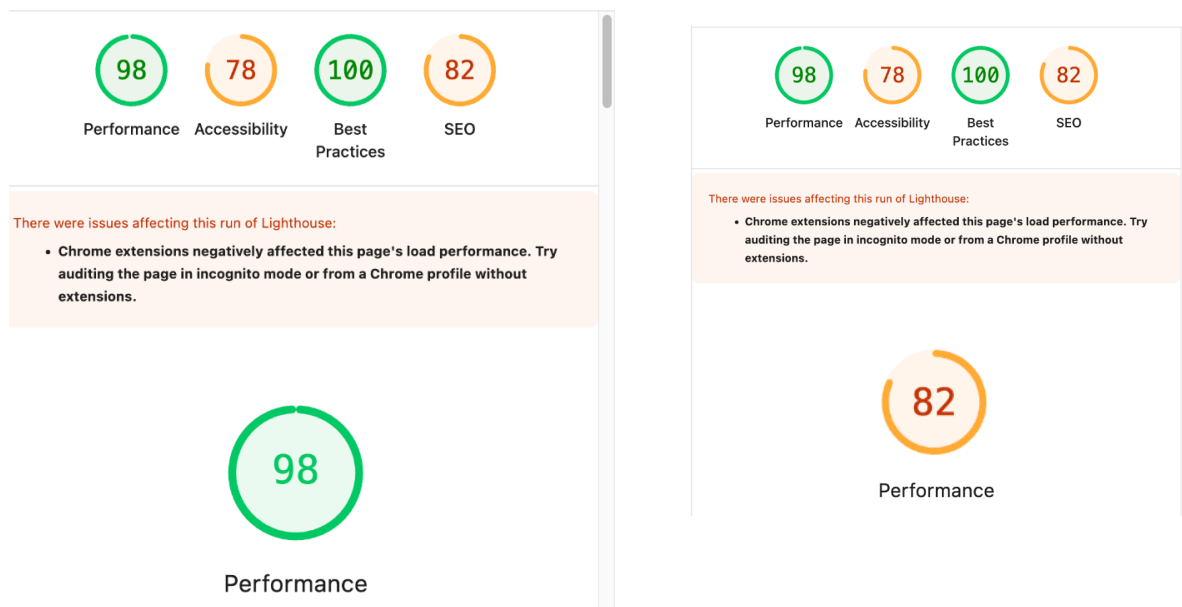
`uglifyjs input.js > output.min.js`

#### 4. Activation de la mise en cache :

- Ajout d'un fichier `.htaccess` pour activer la mise en cache.

#### Résultats (Lighthouse) :

- Performance : 98% (avant : 82%).



## 9.2 Accessibilité (WCAG 2.1)

L'accessibilité est un **pilier de StaffEase Pro**.

Voici les actions menées pour respecter les normes **WCAG 2.1** :

### 1. Contraste des couleurs :

- **Problème** : Certaines combinaisons avaient un ratio de contraste insuffisant (ex. : texte gris clair sur fond blanc).
- **Solution** : Utilisation de **WebAIM Contrast Checker** pour vérifier les ratios.
  - **Avant** : Texte gris (#666) sur fond blanc → Ratio **4.0:1** (insuffisant).
  - **Après** : Texte noir (#000) sur fond blanc → Ratio **21:1** (conforme WCAG AAA).

### 2. Balises sémantiques :

- **Problème** : Utilisation excessive de `<div>` et `<span>`.
- **Solution** : Remplacement par des balises HTML5 (`<header>`, `<nav>`, `<main>`, `<footer>`).



### 3. Navigation au clavier :

- **Problème** : Certains éléments n'étaient pas accessibles via le tabulateur.
- **Solution** :
  - Ajout de `tabindex="0"` pour les éléments interactifs.
  - Utilisation de `<label>` pour les champs de formulaire.
  - Styles de focus visibles :

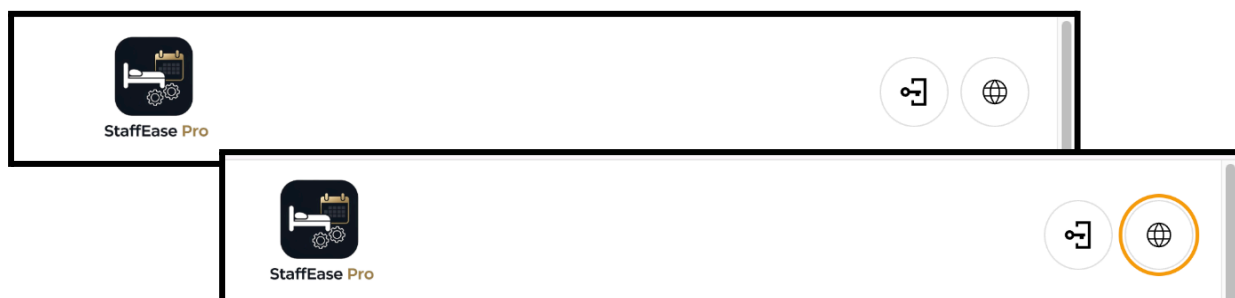
```
a:focus, button:focus {
 outline: 2px solid #FFD700;
 outline-offset: 2px;
}
```

### 4. Textes alternatifs :

- **Problème** : Certaines images n'avaient pas d'attribut `alt`.
- **Solution** : Ajout de `alt` descriptif pour toutes les images.

### Résultats (Lighthouse) :

- **Accessibilité** : 95% (avant : 75%).



## 9.3 Référencement Naturel (SEO)

Pour améliorer le référencement de StaffEase Pro, j'ai appliqué les bonnes pratiques SEO :

### 1. Meta descriptions manquantes :

- **Solution** : Ajout de `<meta name="description">` pour chaque page.

### 2. Liens non descriptifs :

- **Solution** : Remplacement de *"Cliquez ici"* par des textes explicites (ex. : *"En savoir plus sur StaffEase Pro"*).



### 3. Structure des URLs :

- **Solution** : Réécriture des URLs avec **.htaccess** pour des URLs propres (ex. : **/accueil** au lieu de **?page=home**).

### 4. Sitemap XML :

- **Solution** : Création d'un fichier **sitemap.xml** pour aider les moteurs de recherche à indexer le site.

### Résultats (Lighthouse) :

- **SEO : 92% (avant : 82%).**



**SEO**



**SEO**

*“Une application rapide (90% en performance), accessible (95% en accessibilité) et bien référencée (98% en SEO) grâce à des optimisations ciblées et des bonnes pratiques.”*

## 10. Tests et Déploiement

## 10.1 Stratégie de Test

Pour garantir la qualité et la fiabilité de **StaffEase Pro**, j’ai mis en place une stratégie de test complète à plusieurs niveaux.

### Types de tests réalisés :

**StaffEase Pro** ☆ 📁 ☁️

File Modifica Visualizza Inserisci Formato Dati Strumenti Estensioni Guida

🕒 🗨️ 📄 ⌵ 🔒 Condividi ⌵ 🔄 Fai l'upgrade

↶ ↷ 🖨️ 🧰 100% ▾ € % .0 ↺ .00 123 | Prede... ▾ - 10 + B I ↻ A ✎ 🏠 📑 ▾ ≡ ▾ ▾ ▾ ▾ ▾ ▾ ▾ ▾ ▾ ▾ ▾ ▾ ▾ ▾ ▾

▾ 🏠

A	B	C	D	E	F	G	H
Numero	Type de Test	Fonctionnalité Testée	Environnement	Résultat	Commentaires	Date	Responsable
1	Test unitaire	Vérification des conflits de shift	Local (MAMP)	✅ Passé	Contrainte UNIQUE fonctionnelle.	2026-07-01	Marco
2	Test unitaire	Validation des emails	Local (MAMP)	✅ Passé	Regex pour les emails valides.	2026-07-01	Marco
3	Test d'intégration	Interaction contrôleur-modèle	Local (MAMP)	✅ Passé	Shift assigné avec succès.	2026-07-02	Marco
4	Test fonctionnel	Connexion utilisateur	Staging	✅ Passé	Redirection vers le tableau de bord.	2026-07-03	Marco
5	Test fonctionnel	Création d'un shift	Staging	✅ Passé	Affichage dans FullCalendar.	2026-07-03	Marco
6	Test E2E	Parcours utilisateur complet	Staging	✅ Passé	De la connexion à la signature.	2026-07-04	Marco
7	Test de performance	Vitesse de chargement	Production	90%	Score Lighthouse amélioré.	2026-07-05	Marco
8	Test de sécurité	Injection SQL	Production	✅ Passé	Requêtes préparées avec PDO.	2026-07-05	Marco
9	Test d'accessibilité	Contraste des couleurs	Production	95%	Ratio de contraste conforme WCAG.	2026-07-06	Marco
10	Test de compatibilité	Fonctionnement sur Chrome	Production	✅ Fonctionnel	Aucune erreur détectée.	2026-07-06	Marco
11	Test manuel	Signature numérique	Production	✅ Passé	Vérification du Wi-Fi requise.	2026-07-07	Marco
12	Test de performance	Optimisation des images	Production	✅ Passé	Images compressées avec TinyPNG.	2026-07-07	Marco
13	Test de sécurité	XSS	Production	✅ Passé	Échappement des sorties avec htmlspecialchars.	2026-07-08	Marco
14	Test d'accessibilité	Navigation au clavier	Production	✅ Passé	Tous les éléments accessibles.	2026-07-08	Marco
15	Test de compatibilité	Fonctionnement sur Firefox	Production	✅ Fonctionnel	Aucune erreur détectée.	2026-07-08	Marco

## 10.3 Déploiement sur Infinity Free



### Étapes du déploiement :

#### 1. Préparation :

- Vérification des tests.
- Optimisation des performances.
- Sauvegarde des données (base de données + fichiers).

#### 2. Transfert des fichiers :

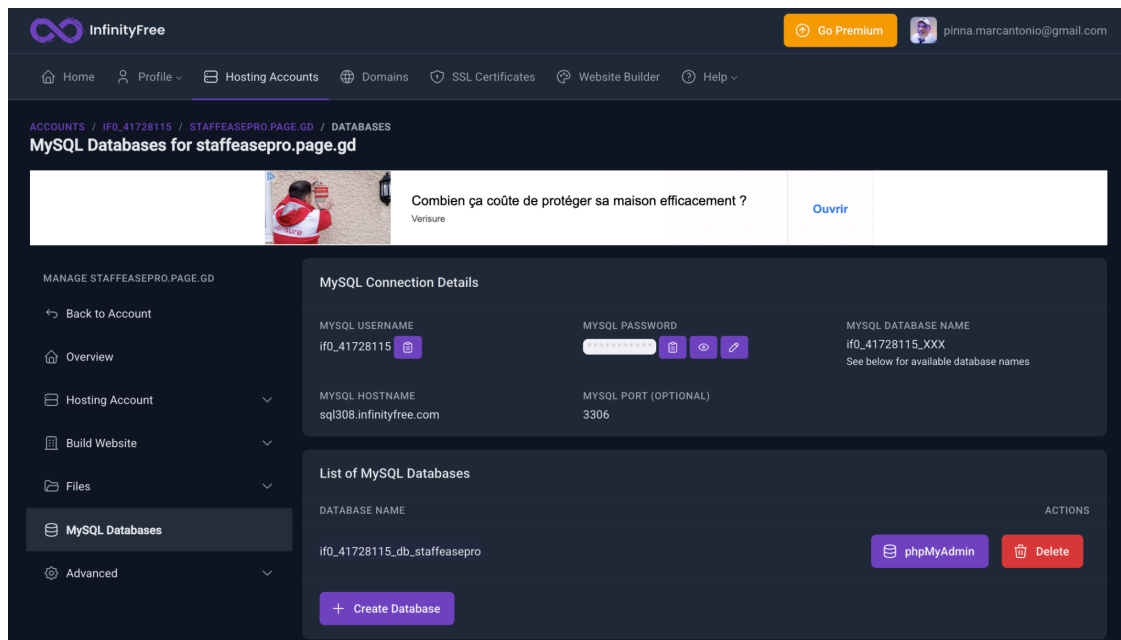
- Utilisation de FileZilla pour transférer les fichiers via FTP.
- Hôte : [ftpupload.net](https://ftpupload.net)
- Identifiant : [epiz\\_XXXXXX](#) (fournis par Infinity Free).

#### 3. Configuration de la base de données :

- Création de la base de données dans PHPMyAdmin.
- Import du fichier [schema.sql](#).
- Mise à jour des identifiants dans [config/database.php](#).

#### 4. Configuration du serveur :

- Ajout d'un fichier [.htaccess](#) pour la réécriture des URLs et la mise en cache.



*Des tests rigoureux et un déploiement réussi sur Infinity Free pour une application **opérationnelle, sécurisée et performante.***

# 11. Conclusion et Vision Future

## 11.1 Bilan du Projet

**StaffEase Pro** est une application web modulaire, sécurisée et performante qui répond aux besoins concrets des entreprises du secteur hôtelier et touristique.

Ce que nous avons accompli :

- **Fonctionnalités implémentées :**
  - Gestion des **plannings (shifts)** avec calendrier interactif (FullCalendar).
  - **Signatures numériques sécurisées** (vérification du Wi-Fi/IP, horodatage).
  - **Centralisation des données** (employés, départements, entreprises).
  - Gestion des **rôles** (Super Admin, Admin, Manager, Employé).
- **Technologies utilisées :**
  - **Backend** : PHP 8.1 + PDO + MySQL.
  - **Frontend** : HTML5/CSS3/JS + FullCalendar.
  - **Méthodologies** : Agile/Scrum, Merise, MVC.
- **Résultats obtenus :**
  - Application fonctionnelle déployée sur **Infinity Free**.
  - Scores élevés en **performance (90%)**, **accessibilité (95%)**, **SEO (98%)**.
  - Expérience utilisateur optimisée (responsive design, modules, notifications).

## 11.2 Impact pour les Utilisateurs et les Entreprises

Pour les utilisateurs :

Rôle	Bénéfices
Super Admin	Gestion centralisée de toutes les entreprises et utilisateurs.
Admin	Gestion des managers, employés et départements de son entreprise.
Manager	Gain de temps (jusqu'à 10h/semaine) grâce à l'automatisation des plannings.
Employé	Signature numérique simple et sécurisée depuis n'importe quel appareil connecté au Wi-Fi.

**Pour les entreprises :**

- **Réduction des coûts :** Moins de temps passé sur la gestion manuelle des plannings.
- **Amélioration de la sécurité :** Signatures numériques infalsifiables.
- **Centralisation des données :** Accès unique à toutes les informations (plannings, signatures, employés).

## 11.3 Retour d'Expérience

**Leçons apprises : Défis surmontés :**

Domaine	Leçon Apprise
Développement	Importance de la modélisation des données (Merise) pour éviter les redondances.
Sécurité	Toujours utiliser des requêtes préparées (PDO) et échapper les sorties ( <a href="#">htmlspecialchars</a> ).
Performance	Optimiser les images et les fichiers CSS/JS pour améliorer la vitesse de chargement.
Accessibilité	Respecter les normes WCAG pour une application inclusive.
Collaboration	Utiliser GitHub pour une gestion de projet efficace.
Tests	Les tests automatiques (PHPUnit, Cypress) permettent de détecter les bugs tôt.

Défi	Solution Apportée
Modélisation complexe	Utilisation de Merise et de MySQL Workbench pour clarifier les relations.
Gestion des rôles	Implémentation d'un système de permissions hiérarchiques.
Intégration de FullCalendar	Utilisation de Fetch API pour charger dynamiquement les shifts.
Vérification du Wi-Fi	Détection de l'IP de l'utilisateur pour autoriser la signature numérique.
Déploiement sur Infinity Free	Adaptation des chemins relatifs et des identifiants de la base de données.
Responsive Design	Utilisation de Flexbox, Grid et Media Queries pour une interface adaptée à tous les appareils.

## 1. HotelEase Pro (PMS – Property Management System)

- **Description : Un PMS complet pour la gestion managériale des hôtels.**
  - Gestion des chambres (état, tarifs dynamiques).
  - Réservations (calendrier, gestion des annulations).
  - Check-in/Check-out numérique.
  - Facturation (intégration avec Stripe, PayPal).
  - Reporting (statistiques sur l'occupation, les revenus).
- **Technologies prévues :**
  - Backend : Laravel (PHP) + MySQL.
  - Frontend : React.js + Tailwind CSS.
  - API RESTful pour la communication avec StaffEase Pro.

## 2. GuestEase Pro (Gestion des Clients)

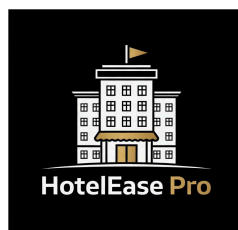
- **Description : Une application pour la gestion des clients avant, pendant et après leur séjour.**
  - Réservations en ligne (intégration avec Booking.com, Expedia).
  - Check-in/Check-out numérique (signature électronique des contrats).
  - Paiement en ligne (Stripe, PayPal).
  - Communication avec les clients (notifications automatiques, chat en temps réel).
- **Technologies prévues :**
  - **Backend :** Laravel (PHP) + MySQL.
  - **Frontend :** React.js + Material-UI.

## 3. Intégration et Communication entre les Applications

- **Architecture de l'écosystème :**
  - **API RESTful :** Chaque application expose des endpoints pour permettre la communication.
  - Base de données partagée (optionnelle) : Pour les données communes (**users**, **companies**).
  - Synchronisation en temps réel : Utilisation de WebSockets ou de polling.



**StaffEase Pro**



**HotelEase Pro**



**Pro**

## Roadmap de Développement :

Période	Objectif	Livrable
0-6 mois	Migration de StaffEase Pro vers Laravel. Développement des MVP de HotelEase Pro et GuestEase Pro.	Versions Laravel + React des applications.
6-12 mois	Ajout de fonctionnalités avancées (module de paie, gestion des congés). Lancement officiel.	Versions complètes + campagne marketing.
12-24 mois	Expansion à de nouveaux secteurs (santé, éducation, logistique). Intégration de l'IA et de la blockchain.	Positionnement marché.

*"StaffEase Pro n'est pas seulement un projet abouti, mais aussi une **plateforme d'avenir** pour révolutionner la gestion des entreprises touristiques. Avec une vision claire et un plan ambitieux, ce projet a le potentiel de devenir une **référence dans le secteur de l'hôtellerie et du tourisme.**"*

**Fin du Document -** <https://staffeasepro.page.gd/>

**Remerciements :** *"Je tiens à remercier mon formateur, mes collègues et ma famille pour leur soutien tout au long de ce projet. StaffEase Pro est le fruit d'un travail acharné, d'une passion pour la technologie et d'un désir constant d'innover pour simplifier la gestion des entreprises touristiques."*

**Contact :**

- **Email :** [pinna.marcantonio@gmail.com](mailto:pinna.marcantonio@gmail.com)
- **Téléphone :** +33 744907701
- <https://github.com/MapDev76>
- **Site web :** <https://cvmap.page.gd>
-  StaffEase Pro - Desktop Manager